Computers & Security 121 (2022) 102828

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

# KRYSTAL: Knowledge graph-based framework for tactical attack discovery in audit data

Kabul Kurniawan[a,c,*], Andreas Ekelhart[b,d], Elmar Kiesling[a], Gerald Quirchmayr[c], A Min Tjoa[e]

[a] Vienna University of Economics and Business, Welthandelsplatz 1, Vienna, Austria
[b] SBA Research, Floragasse 7, Vienna, Austria
[c] University of Vienna, Währinger Straße 29, Vienna, Austria
[d] University of Vienna, Kolingasse 14-16, Vienna, Austria
[e] Vienna University of Technology, Favoritenstraße 9-11, Vienna, Austria

ARTICLE INFO

*Article history:*
Received 10 August 2021
Revised 10 June 2022
Accepted 3 July 2022
Available online 8 July 2022

*Keywords:*
Attack graph construction
Log analysis
Knowledge graph
Attack discovery
Cybersecurity
Information security

ABSTRACT

Attack graph-based methods are a promising approach towards discovering attacks and various techniques have been proposed recently. A key limitation, however, is that approaches developed so far are monolithic in their architecture and heterogeneous in their internal models. The inflexible custom data models of existing prototypes and the implementation of rules in code rather than declarative languages on the one hand make it difficult to combine, extend, and reuse techniques, and on the other hand hinder reuse of security knowledge – including detection rules and threat intelligence. *KRYSTAL* tackles these challenges by providing a knowledge graph-based, modular framework for threat detection, attack graph and scenario reconstruction, and analysis based on RDF as a standard model for knowledge representation. This approach provides query options that facilitate contextualization over internal and external background knowledge, as well as the integration of multiple detection techniques, including tag propagation, attack signatures, and graph queries. We implemented our framework in an openly available prototype and demonstrate its applicability on multiple scenarios of the DARPA Transparent Computing dataset. Our evaluation shows that the combination of different threat detection techniques within our framework improved detection capabilities. Furthermore, we find that RDF provenance graphs are scalable and can efficiently support a variety of threat detection techniques.



## 1. Introduction

In the face of complex cyber attacks, it is crucial to not only detect attacks as early as possible, but also to understand their context and implications in order to choose an appropriate response strategy. To protect themselves, organizations typically rely on defenses such as Intrusion Detection System. These systems are useful in that they can point to indicators of compromise and issues, but they also typically generate a large number of false positive alerts. To identify relevant alerts, it is necessary to investigate their context, which typically requires substantial manual effort and expertise (Hossain et al., 2017; Milajerdi et al., 2019b). As the sheer volume of low-level log data grows, manual analyses become increasingly infeasible.

In this context, system-level provenance graphs, which rely on audit data represented in graph structures, have recently attracted considerable attention in the security research community as a promising tool with strong abstract expression ability and relatively high efficiency (Li et al., 2021). Such provenance graphs represent relationships between the control flow and data flow between subjects (e.g., processes, threads) and objects (e.g., files, network sockets) in the system through a timestamped directed graph. Based on that, a large variety of techniques have been developed to automatically detect and connect attack steps in such graphs.[1]

***Motivating Example*** Figure 1 depicts the provenance graph of an exfiltration attack carried out in the context of the DARPA

* Corresponding author at: Vienna University of Economics and Business, Welthandelsplatz 1, Vienna, Austria.
*E-mail address:* kabul.kurniawan@wu.ac.at (K. Kurniawan).

[1] cf. Li et al. (2021) for a recent survey.

https://doi.org/10.1016/j.cose.2022.102828

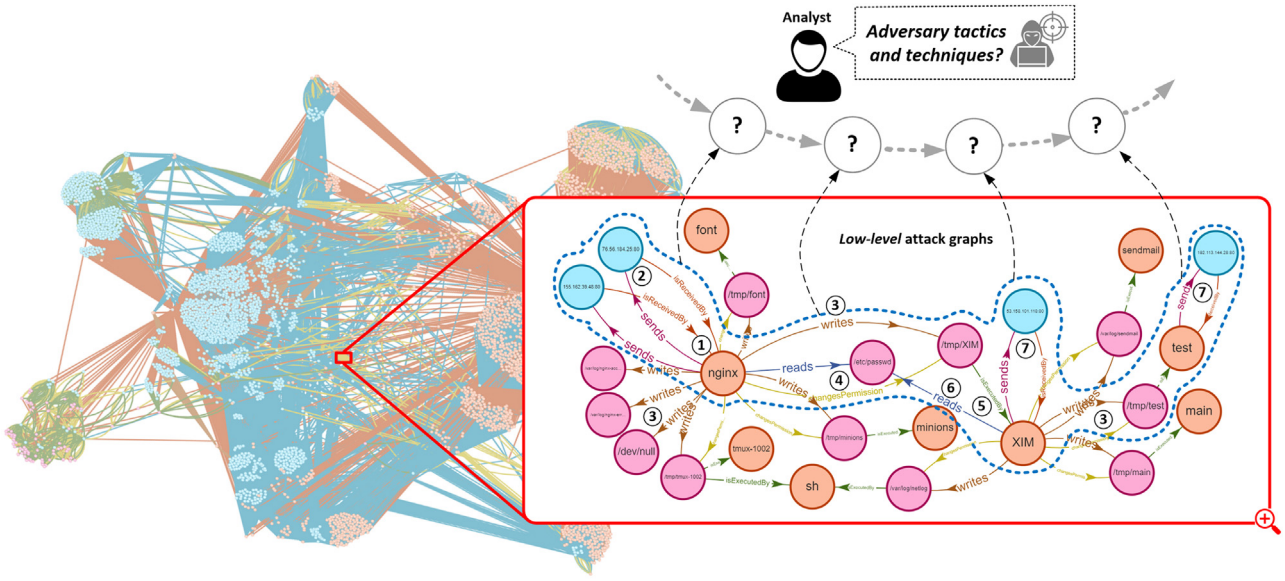


**Fig. 1. Motivating Example** - the enlarged section of the graph (the sub-graph inside the red line) depicts an attack graph. Specifically, the graph inside the blue dotted-line shows *low-level* event interaction as part of the attack pattern. While existing approaches can construct such an attack graph, it remains challenging for an analyst to recognize and understand the real attack steps without linking to *high-level* context (i.e., adversary tactics and techniques). (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Transparent Computing program (DAR, 2021). The depicted graph, which was generated by our prototype, captures running processes, file reads and writes, and sent packets on a host over a given period of time[2] The enlarged section of the graph highlights an attack fragment in which an attacker exploits a vulnerable `nginx` web server through a malformed HTTP request. The attacker opens a shell connection to the victim's host via the vulnerable web server (①②); manages to write an executable file */tmp/XIM* (③④⑤); starts a process that reads sensitive information from */etc/passwd* (⑥), and finally exfiltrates data via HTTP to an external network (⑦). The automatically constructed provenance graph summarizes the attack scenario as a sequence of connected low level events, which provides a good starting point for security analyses.

***Challenges*** Despite its potential, provenance graph-based analysis faces a number of challenges that currently make it difficult to apply them in practical settings.

*Context and interpretability*. The closed nature of existing attack graph-based approaches makes it difficult to relate provenance data to internal and external knowledge. This is crucial because context information is important when interpreting security alerts in order to identify rare occurrences of malicious patterns within an overwhelmingly large amount of activities that are benign.

The graphs typically generated by state of the art techniques connect *low-level* events that are not easily interpretable without additional context. In our motivating example, for instance, the sub-graph inside the blue dotted line shows the low-level events that constitute the sequence of attack (steps ① to ⑦). Without relating such granular low-level events to a *high-level* context, it remains difficult to identify the underlying attack tactics and techniques, interpret the events in a broader context, and *understand the attack*. This lack of abstraction from low-level event graphs to higher level techniques and tactics results in tedious attack investigations necessary to link low-level evidence to phases of complex multi-stage attacks.

*Robust detection*. Provenance graph-based attack discovery approaches have so far been developed as monolithic prototypes implementing a particular combination of techniques. Various prototypes have individually been shown to be effective in identifying and analyzing attacks, but robust detection still remains a key challenge (Li et al., 2021).

*Interoperability*. Existing approaches have so far been developed in the context of tightly coupled research prototypes with proprietary internal data structures that are not interoperable. The respective implementations are typically not openly available, which on the one hand impedes the reproducibility of the findings, and on the other hand hinders integration and reuse.

***Solution approach*** To tackle these challenges, we propose a knowledge graph-driven framework (*KRYSTAL*) that leverages Semantic Web technologies for audit log analysis and tactical attack discovery.

We hypothesize that more robust detection can be achieved in an integrated platform that makes it possible to combine heterogeneous approaches and techniques – which is currently difficult due to the lack of a uniform data model and a common technical foundation. Furthermore, we propose to disentangle data collection, data management, and threat detection – which are currently to a varying degree part of and specific to each approach – by means of a uniform, ontology-based target representation for provenance graphs. This abstracts from storage layer implementation details, facilitates separation of concerns, and provides interoperability between components on these layers. It also makes it possible to reuse and recombine collection, management, and threat detection modules.

To this end, our **contributions** in this paper are as follows: (i) We develop a standardized[3], shared and reusable conceptualization of log events, detection rules, and threat intelligence. Thereby, we unify and integrate log events from heterogeneous sources and enrich it with background knowledge; (ii) We introduce a modular threat detection and attack graph reconstruction framework that leverages this model and integrates multiple state of the art techniques such as tag propagation, attenuation & decay, signature-based, and graph querying; (iii) We generate attack graphs that can be enriched and contextualized through

[2] In this case 140 h.

[3] https://www.w3.org/standards/

**Table 1**
**Comparison of attack discovery approaches**. (*DS*) refers to domain-specific, (*N/A*) refers to not available, (✓) refers to provided, (✗) refers to not provided.

| Approaches | Data Models | Detection Techniques | Data Reduction Techniques | Attack Reconstruction | High-Level Attack Summarization | Background Knowledge Linking | Published Prototype |
|---|---|---|---|---|---|---|---|
| Backtracking (King and Chen, 2003a) | Custom Graph | Naive Backtracking | ✗ | ✓ | ✗ | ✗ | N/A |
| AIQL (Gao et al., 2018b) | Relational DB | DS Query Language | ✗ | ✗ | ✗ | ✗ | N/A |
| SAQL (Gao et al., 2018a) | Relational DB | DS Stream Query | ✗ | ✗ | ✗ | ✗ | N/A |
| POIROT (Milajerdi et al., 2019a) | Custom Graph | Graph Alignment | ✗ | ✓ | ✗ | ✗ | N/A |
| HOLMES (Milajerdi et al., 2019b) | Custom Graph | HSG/Severity Score | ✗ | ✓ | ✓ | ✗ | N/A |
| RapSheet (Hassan et al., 2020) | Custom Graph | EDR/Threat Score | CPR (Xu et al., 2016) | ✓ | ✓ | ✗ | N/A |
| Zou et al. (2020) | Custom Graph | Pre/Post Condition | ✗ | ✗ | ✓ | ✗ | N/A |
| SLEUTH (Hossain et al., 2017) | Custom Graph | Tag Propagation | ✗ | ✓ | ✗ | ✗ | N/A |
| MORSE (Hossain et al., 2020) | Custom Graph | Tag Propagation | CSR (Hossain et al., 2018) | ✓ | ✗ | ✗ | N/A |
| CyGraph (Noel et al., 2016) | NoSQL Graph | DS Query | ✗ | ✓ | ✗ | ✓ | ✓ |
| UCO (Syed et al., 2016) | RDF Graph | SPARQL Query | ✗ | ✗ | ✗ | ✓ | ✓ |
| Ekelhart et al. (2018) | RDF Graph | SPARQL Query | ✗ | ✗ | ✗ | ✓ | ✓ |
| Kurniawan et al. (2020) | RDF Graph | SPARQL Query | ✗ | ✓ | ✗ | ✓ | ✓ |
| SLOGERT (Ekelhart et al., 2021) | RDF Graph | SPARQL Query | HDT (Fernández et al., 2013) | ✗ | ✗ | ✓ | ✓ |
| KRYSTAL | RDF Graph | Hybrid | HDT (Fernández et al., 2013) | ✓ | ✓ | ✓ | ✓ |

linking to background knowledge (e.g., assets, vulnerabilities, cyberthreat intelligence, etc.); such contextualization can help analysts to identify and assess high-level attack scenarios in order to understand their significance, cause, and impact. (iv) We provide an open source prototype of the system[4] and evaluate it on a well-established large-scale dataset (DAR, 2021).

The remainder of this paper is structured as follows: Section 2 discusses related work in threat detection and attack graph construction, Section 3 provides the necessary background on knowledge graph concepts that are central to the developed approach; Section 4 puts forth a set of requirements as a basis of our proposed solution; Section 5 discusses the conceptualization of our approach; Section 6 introduces our knowledge graph-based attack detection framework, and Section 7 discusses the implementation and introduces application scenarios; we evaluate our approach in Section 8, discuss the results in Section 9 and conclude with an outlook on future research in Section 10.

## 2. Related work

In this article we focus on misuse-based intrusion detection techniques, which search for well-defined patterns of attack (Kumar, 1996). Consequently, we organize related work on threat detection and attack graph construction into the following categories: (i) provenance-based tracking, (ii) tactical attack construction, (iii) graph queries, and (iv) ontology-based threat detection. Table 1 summarizes and compares existing attack discovery approaches, which will be discussed in the following sections.

***Provenance-based Tracking*** Seminal work on provenance-based attack detection (King and Chen, 2003a) introduced the idea to investigate attacks through backward tracking. A major limitation of initial "naive" backtracking approaches is that they are based on *coarse-grained* provenance data. This typically introduces a large number of false dependencies in the graph, a problem known as "dependence explosion" (Hossain et al., 2018). Several researchers introduced approaches to mitigate this problem through *fined-grained taint-tracking* or *information flow tracking* (Ji et al., 2017; 2018; Kemerlis et al., 2012). Although these approaches can accurately distinguish suspicious and benign nodes, scaling them remains a challenge (Hossain et al., 2018). Another approach tried to solve this problem by introducing *tag-propagation*. SLEUTH (Hossain et al., 2017), for instance, introduced *trustworthiness tags (t-tags)* and *confidentiality tags (c-tags)* that are used to assign suspicion levels to nodes and propagate them through the provenance graph. In combination with a policy framework, these tags can trigger alarms and successfully identify unseen attacks in real-time and with low overhead. However, SLEUTH suffers from numerous false-positives for attacks with long-running processes (Hossain et al., 2020). MORSE (Hossain et al., 2020) extends this approach by introducing *tag-attenuation* and *tag-decay* to cut down false alarms by more than an order of magnitude. To reduce the provenance graph, it indexes all subjects and objects using a numeric index. However, none of these approaches leverage standard representations, but rely on custom graph models and hard-coded rules and policies. Consequently, they are difficult to expand and it is difficult to investigate the resulting attack graphs further, e.g., by linking them to background knowledge. Furthermore, both SLEUTH and MORSE do not explicitly represent the high-level context of attacks. As a result, it becomes exceedingly difficult to identify attacks as attack graphs become more complex.

Our approach introduces a standard and flexible graph model based on an ontology. Hence, with a standard graph model and declarative rules and policies, our approach can produce compact attack graphs and also flexibly integrate and link them to background knowledge. For graph reduction, we use (Header, Dictionary, Triples) HDT (Fernández et al., 2013), a compact RDF-based structure that keeps large provenance graph compressed. We discuss it in more detail in Section 6.1.

***Tactical Attack Construction*** Provenance-based tracking typically relies on a bottom-up approach, i.e., identifying attacks based on the causality relationship of system objects, such as processes, files, and sockets. By contrast, several approaches follow a top-

[4] https://github.com/sepses/Krystal

down strategy, i.e., they aim to identify attacks based on high-level models of APT campaigns (e.g. techniques, tactics) and/or kill-chain phases. HOLMES (Milajerdi et al., 2019b) introduced threat scores and 16 techniques, tactics, procedures (TTP) proposed by the authors to construct a *high-level scenario graph (HSG)* from a provenance graph. This approach can mitigate dependence explosion and offers a map to TTP. However, its ability to identify attacks with only a single APT stage is limited (Hossain et al., 2020). RapSheet (Hassan et al., 2020) is an attack graph construction approach based on alert correlation from a commercial Endpoint Detection and Response (EDR) tool. By incorporating 67 EDR rules, it can relate alerts into a *tactical provenance graph* based on MITRE's ATT&CK TTPs. For graph reduction, it implements *causality-preserved reduction* (CPR) (Xu et al., 2016) that merges edges with identical operations and keeps only the edge with the latest timestamp. Our approach also facilitates TTP mappings, but we rather incorporate rules from open, standard and community-driven detection rules (i.e., SIGMA Roth and Patzke, 2021), thus avoiding a dependence on a commercial platform.

Another recently proposed approach (Zou et al., 2020) uses tactic-centric Advanced Persistent Threat (APT) recognition to detect APT tactics based on APT techniques' prerequisites (i.e., requirements for techniques to be matched to a tactic) and post-conditions (i.e., the result of a technique, e.g., malicious process being created). The identified attack techniques are mapped to specific tactics and ranked based on their tactic matching. This approach shows the detected tactics and techniques, but it does not represent the complete attack scenario, i.e., attack sequences, causality and connections.

In our work, we propose a hybrid approach that facilitates both bottom-up and top-down techniques for threat detection and attack construction in a single, modular framework. Our approach generates detailed attack graphs from low-level events but also facilitates linking and contextualization to existing high-level, tactical attack patterns such as MITRE ATT&CK TTPs.

***Graph Queries*** A number of research efforts resulted in query-based and graph-matching approaches to detect and construct attack scenario graphs from historical audit log data stored in databases. AIQL (Gao et al., 2018b) introduced a domain-specific model and query language to analyze and investigate attacks. SAQL (Gao et al., 2018a) extends AIQL for stream-based querying over system monitoring data. POIROT (Milajerdi et al., 2019a) proposed an attack graph detection approach based on manually extracted graph patterns from previously seen attacks, e.g., in threat intelligence reports. In our approach, we rely on a semantic model, which facilitates graph matching through semantic graph queries (formulated in SPARQL). Furthermore, our RDF-based provenance graph can be easily queried and linked to internal and external background knowledge through SPARQL query federation.

***Ontology-based Threat Detection*** A number of research efforts investigated ontologies to support cybersecurity and threat detection. Early work (Pinkston et al., 2003) developed an ontology for the intrusion detection domain based on DAML+OIL (Mcguinness et al., 2002) to extend simple IDS taxonomies with machine-interpretable definitions. Ref. More et al. (2012) extended this IDS Ontology to incorporate cybersecurity-related information from heterogeneous resource (e.g., web texts, reports).

Unifie Cybersecurity Ontology (UCO) (Syed et al., 2016) introduced a more instance-data driven approach to construct a rich cybersecurity ontology by integrating cybersecurity standards such as STIX (2021), CyBox (2021), CVE (2021), CAPEC (2021), CVSS (2021a), and CVSS (2021b). UCO provides integration from heterogeneous sources and supports reasoning (e.g. using predefined rules to infer attacks). However, UCO and other previously proposed ontologies were designed to support intrusion detection rather than attack graph construction. More recent works such as Kenaza and Aiash (2016) proposed an ontology that supports IDS alert correlation. Through the use of reasoning rules, it can infer and classify IDS alerts (i.e., false alert, unclassified, plausible attack) based on existing vulnerability information. As the focus is solely on attack detection, however, this proposed approach does not result in attack graphs.

In prior work, we proposed a modular ontology (Ekelhart et al., 2018) that can harmonize and integrate heterogeneous log sources (e.g., Syslog, Auth log, Apache log). Subsequent work (Kiesling et al., 2019) facilitates forensic analyses of arbitrary logs and contextualizes them with external background knowledge (e.g. CPE, CVE, CVSS, CWE, and CAPEC) in a federated settings (SPARQL, 2021). SLOGERT (Ekelhart et al., 2021) extends this previous work with the ability to construct knowledge graphs automatically from arbitrary raw log messages. It identifies, links, and enriches entities in log sources with background knowledge. In this work, we extend and enhance our previous work on security ontologies to integrate events from heterogeneous log sources, represent them in attack scenario graphs, and link them to background knowledge.

## 3. Background

In this section, we introduce knowledge graphs as a conceptual foundation of our approach.

***Knowledge Graph*** A knowledge graph is a directed, labeled graph $G = (V, E)$ where $V$ is a set of vertices (nodes) and $E$ is a set of edges (properties). A single graph is usually represented as a collection of triples $T = < s\ p\ o >$ where $s$ is a *subject*, $p$ is a *predicate*, and $o$ is an *object*.

***RDF, RDF-S, OWL*** Resource Description Framework (RDF) is a W3C[5] standard data model to represent knowledge graphs. In RDF, a *subject* is a resource identified by a unique identifier (URI) or a blank-node, an *object* can be both resource, blank-node or literal (e.g. String, number), and *predicate* is a property defined in an ontology and must be a URI. RDF-S (Resource Description Framework Schema)[6] is a W3C standard data model for knowledge representation. It extends the basic RDF vocabulary with a set of classes and RDFS entailment (inference patterns).[7] OWL (Ontology Web Language)[8] is also a W3C standard for authoring ontologies. We use RDF-S and OWL to represent our system-call provenance ontology. We discuss this in more detail in Section 5.2.

***Semantic Reasoning*** One of the central ideas of both OWL and RDF-S is that they enable reasoning over a knowledge graph. This is an important aspect and enables us to automatically infer new knowledge based on existing facts (entities and their relationships). We leverage several built-in RDF-S and OWL reasoning rules in our approach as discussed in Section 5.3, which are based on the foundation of *description logics* (Horrocks, 2005). Furthermore, in our approach, semantic reasoning is used to simplify RDF mappings during the provenance graph building process (discussed in more detail in Section 6.1) and to reduce the query complexity, e.g., for attack reconstruction i.e. *backward-forward chaining* mechanism (discussed in Section 6.3).

***SPARQL*** SPARQL[9] is a W3C query language to retrieve and manipulate data stored in RDF. It offers rich expressivity for complex queries such as aggregation, subqueries, and negation. Furthermore, SPARQL provides capabilities to express queries across multiple distributed data sources through SPARQL query federa-

[5] https://www.w3.org/standards
[6] https://www.w3.org/TR/rdf-schema/
[7] https://www.w3.org/TR/rdf11-mt/
[8] https://www.w3.org/TR/owl2-overview/
[9] https://www.w3.org/TR/sparql11-overview/

tion.[10] In the security context, this is a major benefit, as security-relevant information is typically dispersed across different systems and networks and requires the consideration of e.g., different log sources, IT repositories, and cyberthreat intelligence sources (Kurniawan et al., 2020).

## 4. Requirements

Based on our analysis of the limitations of the state of the art (cf. Section 2), we identify the following set of requirements for the construction of a modular framework for knowledge graph-driven tactical attack discovery.

***R1. Contextualization***
Investigating and prioritizing security alerts, which typically include a large number of false positives, requires extensive security domain knowledge to understand the context of an alert (Sarker et al., 2020). This makes it necessary to contextualize information in the provenance graph with appropriate background knowledge, which necessitates a flexible data model. As an example, important contextual information on the system under evaluation includes installed software and patch versions of the host where an alert has been raised. Existing approaches (e.g., Hossain et al., 2020; Milajerdi et al., 2019a) typically hard-code some context information (e.g., external networks and software), whereas the majority currently ignores them altogether. To overcome these limitations, the ability to contextualize provenance graphs with background knowledge on the system under investigation is a key requirement for our framework.

***R2. Reusability and Extensibility*** Reusability – e.g., of attack patterns and resulting graphs – and extensibility – e.g., of detection, reconstruction, and summarization techniques – have not been design priorities in existing monolithic solutions for provenance based log analysis. Consequently, approaches each rely on their own data structures developed specifically for each approach, While this allows for some optimization, it makes it difficult to reuse and extend methods, techniques, information, and results. The lack of unified data formats as a (more specific) aspect of this requirement has also been highlighted as a major limitation of the state of the art in a recent survey (Li et al., 2021). To tackle this limitation, the developed framework should – while offering adequate performance – make it possible to: (i) formulate and exchange rules for detection, alerting, and attack graph reconstruction in a declarative language, (ii) exchange instances of provenance graphs in a standard representation, (iii) query the provenance graphs in a standard language, (iv) incorporate and exchange threat information, and (v) reuse and combine analytic techniques.

***R3. Threat intelligence linking*** Connecting isolated events to reconstruct a complete attack scenario is an important step to understand the relevance and impact of alerts. In this context, an ability to link low-level threat evidence to phases of complex multi-stage attacks would be highly beneficial. While some existing approaches (Hassan et al., 2020; Milajerdi et al., 2019b; Zou et al., 2020) aim to identify steps and associate them with high-level APT phases, they do not take advantage of the abundance of available Cyber Threat Intelligence (CTI) information available in external sources such as MITRE ATT&CK (Mitre, 2021), which offers links to Common Vulnerabilities and Exposures (CVE) (CVE, 2021), Common Weakness Enumeration (CWE) (CWE, 2021), and others. We therefore define the ability to leverage such connections and provide integrated querying capabilities as a key requirement for the developed approach. This will facilitates abstraction from low-level event graphs to higher level techniques and tactics, provide additional information for prioritization, impact assessment and mitigation, and make tedious attack investigations more efficient and effective.

***R4. Cross-platform Interoperability*** Integrating provenance data from heterogeneous sources and constructing integrated attack graphs spanning multiple systems would be highly beneficial in the face of complex multi-host/multi-platform attacks (Li et al., 2021). This necessitates a unified provenance representation that can abstract from specifics of individual platforms.

## 5. Conceptualization

In this section, we describe the conceptualization of our approach and the use of inference for provenance graph construction.

### 5.1. System-Call provenance representation

Provenance graphs are a highly effective representation to keep track of information flows (Li et al., 2021). They represent interactions between objects as events. Each event involves a *subject*, i.e., a system object (e.g., a process, thread) that performs a particular *operation* (e.g., write, read, send) on an *object* (e.g., file, socket, registry). As objects typically appear in multiple events, a graph emerges. Although system-call provenance is represented similarly across different existing approaches (Hassan et al., 2020; Hossain et al., 2017; 2020; Milajerdi et al., 2019b), no formal and standardized representation (ontology) exists. Instead, existing approaches typically use ad-hoc and hard-coded data models which limits their interoperability, reusability and extensibility.

### 5.2. KRYSTAL *Provenance ontology*

We propose the *KRYSTAL* ontology[11] as a standard representation for system-call provenance graphs. Key benefits of using ontologies is the ability to share a common understanding – including concepts and structure, making assumptions explicit, facilitating semantic reasoning, and promoting concept reuse (Noy and McGuinness, 2001). Furthermore, the use of an ontology for provenance graph representation makes it easy to integrate background knowledge and unify the conceptual model across different threat detection approaches.

In developing the *KRYSTAL* ontology, we followed a *hybrid* approach, i.e., using *bottom-up* and *top-down* approaches concurrently (Noy and McGuinness, 2001). Bottom-up, we started by analysing low-level data structures from applications (e.g., auditlog) and identifying their entities and relations. Top-down, we considered attack patterns from existing cyberthreat intelligence sources (e.g., MITRE ATT&CK Mitre, 2021) and compared our ontology to existing non-ontological provenance models/schemas. Our developed ontology is able to represent low-level system-call provenance data and link it to high-level attack patterns (cf. Section 6.4).

Figure 2 depicts an excerpt of the *KRYSTAL* Ontology centered around core concepts such as `User`, `Host`, `System Object`. The latter represent system entities such as `Processes`, `Files`, and `Sockets`. We assign each system object *so* to a class (identified by `rdfs:Class`) and potential sub-classes (identified by `rdfs:subClassOf`). Each system object *so* is a vertex $v$ and the relations between system objects (e.g., `writes`, `isReadBy`, `sends`, etc.) are represented via edges $e$ (identified by `owl:ObjectProperty`).

The SystemObject class represent system entities in general. It has three sub-classes such as Process, File and Socket. The Process class represents a running process in a system (e.g. firefox, ssh, etc.) while the File class represents a file (e.g., system file, application file, etc.). We introduce `kry:writes`, `kry:isReadBy`

[10] https://www.w3.org/TR/sparql11-federated-query/

[11] https://w3id.org/sepses/vocab/event/log/

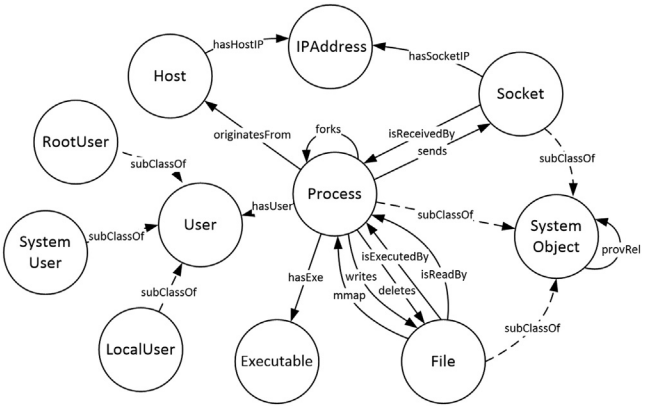


**Fig. 2. Krystal Ontology** - nodes represent concepts (classes) and edges represent the class relationship (properties).

**Table 2**
RDFS & OWL Reasoning Rules.

| Rule | Premise | Conclusion |
|---|---|---|
| **rdfs2** | $e_1(v_1, v_2)$, `rdfs:domain`$(e_1, X)$ | $v_1 \in X$ |
| **rdfs3** | $e_1(v_1, v_2)$, `rdfs:range`$(e_1, Y)$ | $v_2 \in Y$ |
| **rdfs7** | $e_1(v_1, v_2)$, `rdfs:subPropertyOf`$(e_1, e_2)$ | $e_2(v_1, v_2)$ |
| **owl: InverseOf** | $e_1(v_1, v_2)$, `owl:inverseOf`$(e_1, e_2)$ | $e_2(v_2, v_1)$ |

and `kry:isExecutedBy`, `kry:deletes`, `kry:mmap` properties to represent links between PROCESS class and FILE class. The SOCKET class represents a network connection (combination of an `IPAddress` and `Port`). We defined `kry:sends` and `kry:isReceivedBy` properties that link the PROCESS class to the SOCKET class.

We also defined USER, a class that describes user of a system. It has three sub-classes, i.e., (i) ROOTUSER, that represents the root-level user, (ii) SYSTEMUSER, represents the system-level user, (iii) LOCALUSER, represents the local-level user.

We also defined HOST class that represent a host machine. The `kry:hasUser` property connects PROCESS class to the USER class, while the `kry:originatesFrom` property links it to HOST class. Finally, we also defined several other classes such as EXECUTABLE class that identifies an executable file and IPADDRESS that represents an IP Address. The `kry:hasExe` property connects PROCESS to EXECUTABLE and `kry:hasHostIP` that links PROCESS to IPADDRESS.

Furthermore, we introduce a *provenance relation* (identified by `kry:provRel`) that self-links to SYSTEMOBJECT. It is a generic, upper-level relation property that is used to represent provenance relationships between system objects (e.g., socket to process, process to file, file to process). This provenance relation is automatically inferred from more specific relation properties, as explained in the following.

### 5.3. Inference features in KRYSTAL ontology

The *KRYSTAL* ontology uses the RDF-S entailment and OWL reasoning rules summarized in Table 2 for provenance graph construction.

***rdfs2 & rdfs3*** These rules are used to automatically infer the specific type of a given system object based on its relations.[12]

For example, as depicted in Fig. 3, system object `:Firefox`[13] has the relation `kry:writes` to another system object

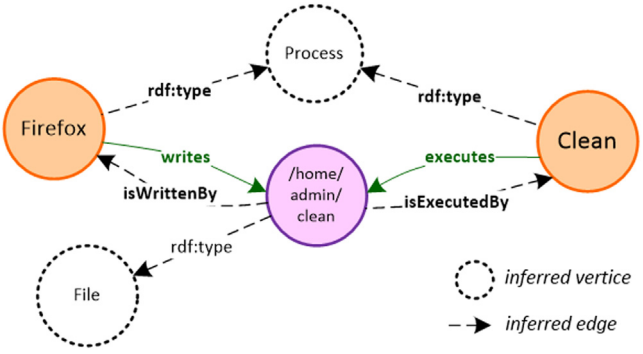


**Fig. 3.** RDFS & OWL reasoning for vertice/edge inference.[12]

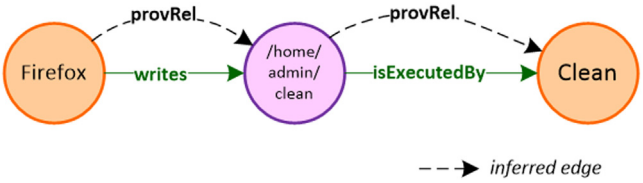


**Fig. 4. rdfs7** (`rdfs:subPropertyOf`) inference example.

`:/home/admin/clean`, since in the ontology we define `kry:writes` as `owl:ObjectProperty` with an `rdfs:domain` of `kry:Process`. Consequently, based on **rdfs2** rule definition, it will be automatically inferred that `:Firefox` $\in$ `kry:Process`. The system object `:/home/admin/clean` is another example. Since `kry:writes` has `rdfs:range` of `kry:File`, based on the **rdfs3** rule definition, it will be deduced that `:/home/admin/clean` $\in$ `kry:File`. This class type inference is useful for, e.g., formulating semantic graph queries to detect attack patterns based on the system object type and their relations. We discuss the query processes in more detail in Section 6.

***rdfs7*** This rule is used to generalize relations based on property hierarchies identified by `rdfs:subPropertyOf`. In the example in Fig. 4, for instance, `:Firefox` has the relation `kry:writes` to `:/home/admin/clean`, and in the ontology, `kry:writes` is a sub-property of the more generic property `kry:provRel`, which represents provenance relationships.

Based on the **rdfs7** rule, `kry:provRel` will be automatically inferred between objects that have a more specific relation; this creates provenance links for all system object relationships (e.g., from the `:Firefox` process to the `:/home/admin/clean` file to the `:Clean` process etc.). This is helpful to identify information flows, track causality of events, and reconstruct attack graph. We discuss the advantage of having explicit provenance relations (e.g., during backward-forward analysis for attack graph reconstruction) further in Section 6.3.

***owl:InverseOf*** Finally, a reasoning technique that we use in our ontology is property inversion, identified by **owl:InverseOf**, which creates relations between system objects in both directions. For instance, as depicted in Fig. 3, `:Firefox` has a relation `kry:writes` to `:/home/admin/clean` and in the ontology, `kry:writes` is defined as `owl:inverseOf` to `kry:isWrittenBy`. Based on the **owl:inverseOf** rule, it will be automatically inferred that `:/home/admin/clean` also has a relation `kry:isWrittenBy` to `:Firefox`. The same holds for property `kry:executes`, since it has the relation `owl:inverseOf` to `kry:isExecutedBy` in the ontology.

## 6. Solution architecture

In this section we present *KRYSTAL*, a modular framework for tactical attack discovery in audit data. The proposed framework in-

[12] We represent objects with different colors: orange for processes, purple for files, blue for sockets, and transparent-dotted circles for inferred system object types.

[13] *Note that we define prefix ":" for an instance and "kry:" for Krystal ontology. We omit it in figures for simplicity.*

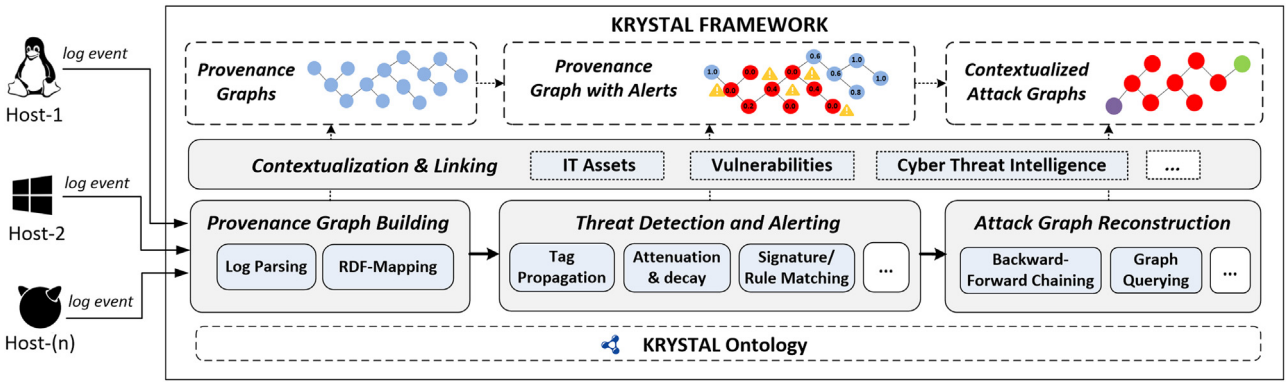


**Fig. 5.** Krystal Framework Architecture.

```
@prefix : <http://w3id.org/sepses/resource#> .
@prefix kry: <http://w3id.org/sepses/vocab/kystal#> .

:Firefox kry:writes :/home/admin/clean ;
    kry:cmdLine "/usr/bin/firefox" ;
    kry:hasUser :user-2 ;
...
```

**Listing 1.** Excerpt of an RDF-based provenance graph generated from a log event.

```
@prefix : <http://w3id.org/sepses/resource#> .
@prefix kry: <http://w3id.org/sepses/vocab/kystal#> .
...
:Firefox a kry:Process, kry:SystemObject;
    kry:provRel :/home/admin/clean.

:/home/admin/clean a kry:File, kry:SystemObject.
...
```

**Listing 2.** Excerpt of inferred RDF triples.

tegrates a variety of attack discovery mechanisms and takes advantage of its semantic model to include internal and external knowledge in the analysis. Fig. 5 gives an overview of the KRYSTAL attack discovery framework which consists of three main parts, i.e., (i) provenance graph building, (ii) threat detection and alerting, and (iii) attack graph and scenario reconstruction.

Security analysis is typically conducted either *online* or *offline*. Online refers to an analysis performed in a running system in (near) real-time, whereas offline refers to analysis over collected data. These two modes have a different purpose (monitoring vs forensics), but they can complement each other in hybrid settings (Marz and Warren, 2015).

*KRYSTAL* works in an online mode in that it imports each log event in sequence from potentially heterogeneous hosts (e.g., Linux, Windows, FreeBSD). The *Provenance Graph Building module* then generates an RDF-based provenance graph, taking advantage of the well-defined ontology and enabling enrichment with background knowledge. Subsequently, the *Threat Detection & Alerting* module allows for the combination of various approaches on the uniform Knowledge Graph (KG). We illustrate the generality of the approach by implementing a set of common mechanisms as SPARQL queries, combining (i) tag propagation, (ii) attenuation & decay, and (iii) signature-based detection based on Indicator of Compromise. The *Attack Graph Reconstruction* module then facilitates (offline) attack graph generation via *Backward-forward* chaining and attack pattern matching via *Graph Querying* over the provenance graph. We explain each component in more detail in the following subsections.

### 6.1. Provenance graph building

This component consists of two sub-components, *Log Parsing* and *RDF Mapping*. *Log Parsing* transforms raw log events from heterogeneous hosts and operating systems (e.g., `auditd` from Linux, `ETW` from Windows, and `dtrace` from FreeBSD) into a structured format (i.e., JSON). This component selects and parses important information from the log event such as system entities (e.g., processes, files, sockets, etc.) and their relations (e.g., read, write, execute, etc). *RDF-Mapping* maps the structured log data into an RDF graph representation. Specifically, we used the *KRYSTAL* ontology described in Section 5.2 and RML[14], a declarative RDF-mapping language, to map the parsed audit data and transform them into well-defined, RDF-based provenance graphs.

Listing 1 shows an excerpt of a log event in RDF representation. It captures the fact that a system object (Firefox process) created a file ("/home/admin/clean"). In RDF, the Firefox process is modeled as *subject* `:Firefox`, the "write" operation is represented as *property* `kry:writes` and the file "/home/admin/clean" is connected as *object* `:/home/admin/clean`.

Taking advantage of reasoning rules (cf. Section 5.3), Listing 2 shows the inferred RDF triples from the provenance graph. `:Firefox` has been identified as a `kry:Process` and `kry:SystemObject`. In addition, the provenance relation *property* `:provRel` with the *object* `:/home/admin/clean` has been added automatically, and the type of `:/home/admin/clean` has been inferred as `kry:File`.

***Graph Reduction & Compression*** We apply two strategies to reduce the graph size. First, we automatically merge the duplicated RDF output identified by the same URI (2021), thus eliminating redundant events (events with the same *subject*, *property* and *object*). Furthermore, we skipped irrelevant events – i.e., events that are not considered in our ontology – from being processed. Second, we use Header Dictionary Triples (HDT) (Fernández et al., 2013) as a graph compression technique that provides a compact data structure and binary serialization format for RDF. HDT keeps large graph data compressed and manageable while enabling query operations without prior decompression. We discuss our graph reduction and compression results in Section 8.

### 6.2. Threat detection & alerting

Due to the uniform KG representation, the *KRYSTAL* framework allows to combine and integrate a variety of techniques for threat detection and alerting. We illustrate this by (i) transforming and

[14] https://rml.io/

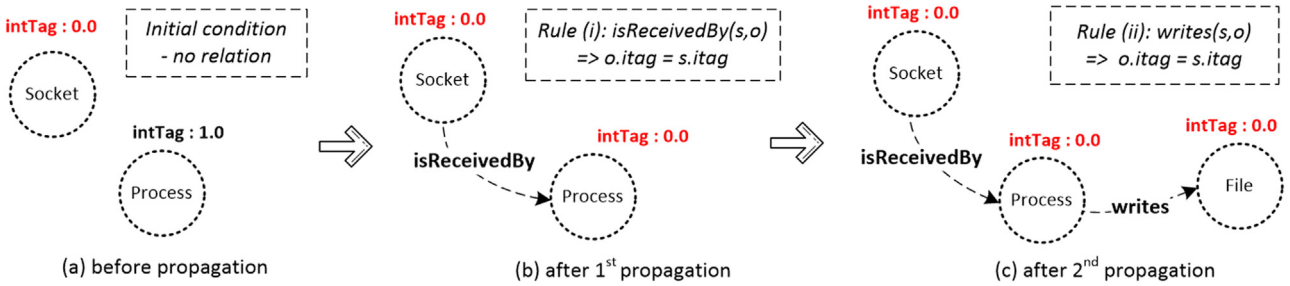


**Fig. 6.** Tag Propagation Example.

```
PREFIX : <http://w3id.org/sepses/vocab/krystal#>
PREFIX afn: <http://jena.apache.org/ARQ/function#>

DELETE { <process> :intTag ?sit }
INSERT { <process> :intTag ?nit }
 WHERE { <socket> :intTag ?oit;
                  :isReceivedBy <process>.
         <process> :intTag  ?sit.
         FILTER (?oit != ?sit).
         BIND (afn:min(?oit,?sit) AS ?nit). }
```

**Listing 3.** Propagation rule for incoming socket connections.

```
PREFIX : <http://w3id.org/sepses/vocab/krystal#>
PREFIX afn: <http://jena.apache.org/ARQ/function#>

DELETE {<file> :confTag ?oct}
INSERT {<file> :confTag ?noct}
WHERE  {<file> :confTag  ?oct.
        <process> :writes <file>; :confTag  ?sct.
        FILTER (?oct != ?sct).
        FILTER (?oct != ?noct).
        BIND (?sct + 0.2 AS ?nsct).
        BIND (afn:min(?oct,?nsct) AS ?noct). }
```

**Listing 4.** Attenuation rule for a benign write propagation.

integrating established threat detection and alerting techniques (ii) incorporating a signature-based threat detection approach through the transformation of curated public rules (e.g. Sigma Rule), and (iii) link the identified attack pattern into high-level attack technique and tactic (TTPs Mapping). Rather than hard-coding these mechanisms and developing them for a custom data structure, we show in the following how they can be implemented as standard declarative SPARQL queries. ***Tag Propagation*** is a prominent method to establish event causality in the context of provenance graphs. It is based on a set of rules to assign tag values onto system object nodes (i.e., process, file, socket, etc.) upon interaction between them (Hossain et al., 2017). In order to trace the impact of malicious events, tags and values will be propagated sequentially through a provenance graph to other system objects if a given *tag propagation* rule is satisfied.

Figure 6 illustrates *tag propagation* with an example. It introduces *integrity* and *confidentiality* tags to identify the suspicion level of a node. We express propagation rules Hossain et al. (2020) as SPARQL queries processed by our *tag propagation* component. Listing 3 shows the query matching the illustration in Fig. 6. It checks if a socket connection (`<socket>`) is received by (`:isReceivedBy`) a process (`<process>`) and then compares the minimum (`afn:min`) integrity tag value (`:intTag`) of the `<socket>` (identified by `?oit` variable) and `<process>` (identified by `?sit` variable).[15] If `?oit` is not the same as `?sit`, it updates[16] `?sit` with the new value (identified by the `?nit` variable).

***Attenuation & Decay*** are techniques to tackle the "dependence explosion" problem (Hossain et al., 2017), which occurs when a node in the provenance graph interacts with a large number of system objects, causing a large number of benign events to be flagged as being part of an attack (Hossain et al., 2020). This leads to a large number of false-positive alerts, making it difficult to identify relevant alerts. *Tag attenuation* (Hossain et al., 2020) aims to alleviate this issue by considering objects imperfect intermediaries for propagating malicious behavior through a benign process. In the previous example (cf. Fig. 6), for instance, the low integrity process $p_1$ writes a file $f_1$, lowering its integrity. To avoid excessive propagation to other objects from there, the integrity and confidentiality tags of a benign subject get attenuated before they propagate to another benign object. This can be achieved by applying an additive factor *af* to the original tag value.

*Tag decay* is based on the assumption that in case a benign subject gets compromised and becomes suspicious, it will do so soon after consuming a suspicious input (e.g., read low integrity file) (Hossain et al., 2020). Consequently, this technique gradually lifts the score of the low integrity subject and limits benign objects from being propagated and flagged as suspicious, particularly for long-running processes.

Listing 4 illustrates how the *KRYSTAL* framework enables declarative definitions of such rules using SPARQL queries. It shows an example of an attenuation rule for a benign write propagation that checks for a `<process>` that `:writes` a confidential `<file>`. The rule gradually increments the confidentiality tag (`:confTag`) value (`?sct`) of processes.

***Provenance-Based Alerting*** Provenance-based alerting policies detect attacks based on the simultaneous fulfillment of several conditions in the provenance graph. These conditions (cf. Hossain et al., 2020) take data integrity tags as well as information associated with nodes (e.g., permissions, users) into account. For example, a suspicious file execution can be detected under the following alert policy:

- a file $f$ is executed by a process $p$,
- $f$ has a low integrity tag value ($<0.5$) and $p$ is benign (integrity $>=0.5$).

The *KRYSTAL* framework facilitates provenance-based alerting by expressing alert policies as SPARQL queries and matching them against the provenance log graph. Listing 5 provides an example of an alerting policy for suspicious file execution. The query matches a file (`?file`) with a low integrity value that has been executed by a benign process. Specifically, files with a tag value

[15] System objects with integrity scores in the interval $[0.0 - 0.5|$ are considered suspicious and scores in the interval $[0.5 - 1.0]$ are considered benign.

[16] SPARQL uses `DELETE` and `INSERT` to perform update operations on triple(s); cf. https://www.w3.org/TR/sparql11-update/

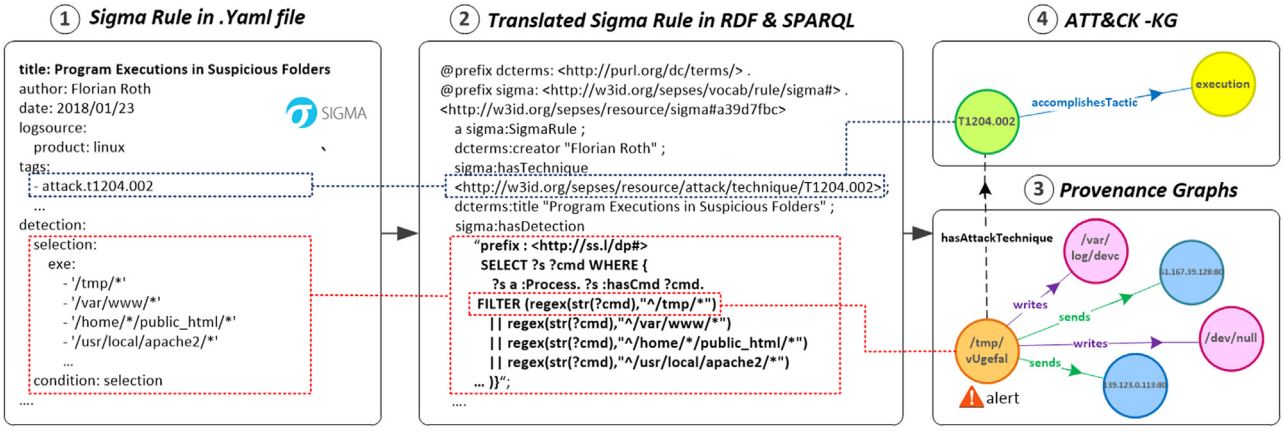


**Fig. 7.** Signature/rule-based threat detection example using the translated *Sigma*-rule query.

```
PREFIX rule: <http://w3id.org/sepses/vocab/rule#>
PREFIX : <http://w3id.org/sepses/vocab/krystal#>
CONSTRUCT {
       << ?file :isExecutedBy ?process >>
          rule:hasDetectedRule rule:execRule }
 WHERE { ?file :isExecutedBy ?process.
         ?file rule:intTag  ?oit.
         ?process rule:subjTag ?sst.
         FILTER (?oit < 0.5)
         FILTER (?sst >= 0.5)}
```

**Listing 5.** Alerting policy represented as SPARQL query.

below 0.5 are considered low integrity and the respective triple `?file :isExecutedBy ?process`) will be marked with the detected rule.[17]

***Signature/Rule-based Threat Detection*** We complement the provenance-based mechanisms with a signature/rule-based detection approach that utilizes IoC definitions to identify known attacks in log events. This illustrates that the uniform KG representation facilitates the combination of a variety of approaches by using a common declarative query language.

Signatures are an established and effective approach in detecting known attack patterns, but maintaining the set of rules and signatures can be labour-intensive (Li et al., 2021). To tackle this problem, *KRYSTAL* leverages *Sigma*[18] – an open, shareable, community-driven and generic rule format for threat detection in logs.

Figure 7 part (1) shows an example of signature/rule-based threat detection defined in *Sigma*.[19] Each *Sigma*-rule is written in YAML[20] and defines the detection rule and its metadata (*title, id, date, author, log-source* etc.). A key benefit of these rules is that the *tags* metadata links rules to specific TTPs from MITRE ATT&CK. For example, *attack.t1204.002* corresponds to the technique *T1204.002*[21] (*User Execution: Malicious File*). As per November 2021[22], *Sigma* contains more than 1497 rules/signatures for different log sources (e.g., application, network, web logs) and platforms (e.g., Linux & Windows).

Our framework translates these *Sigma* rule specifications[23] automatically and transforms them into SPARQL query expressions. Specifically, we identified two search-identifiers under the *detection* attribute: (i) *lists* that contain strings applied to the full log message and joined with a logical 'OR', and (ii) *maps* that consist of key/value pairs, where key is a field in the log data and value is a string or integer. Lists of maps are joined with a logical "OR" while all elements of a map are joined with a logical "AND".

We express *lists* as SPARQL filters with regex matching, while *maps* are represented as triple patterns, i.e. *Subject (S), Predicate (P), and Object (O). S* is a log object, *P* is a log property i.e., key/field in the log data and *O* is the value. Similar to *lists*, we express the key/value pair matching using regex filters in SPARQL. Furthermore, we also map and transform the rule metadata into RDF. Figure 7 part (2) shows an example translation of a *Sigma* rule into SPARQL and RDF.

The translated *Sigma* rules are executed against the provenance graph to detect potential attacks/threats in the *Threat Detection and Alerting* module of our framework. As we can see in Fig. 7 part (3), we detect an alert called "*Program Executions in Suspicious Folder*" in the provenance graph since the execution of the */tmp/vUgefal* process is located in the */tmp/* folder as defined in the *Sigma* rule. Subsequently, the generated alert will be linked automatically to the T1204.002 (*User Execution: Malicious File*) technique from the ATT&CK knowledge graph (Kurniawan et al., 2021) in the background knowledge (cf. Fig. 7 part (4)). We explain this linking mechanism further in Section 6.3.

### 6.3. Attack graph reconstruction

The next important step once the provenance graph has been constructed and alerts have been raised is to understand how the alerts are connected and to reconstruct potential attack steps. To this end, we construct an attack graph and the attack scenario out of the provenance graph through (i) *Backward-Forward Chaining* and (ii) *Graph Querying*.

***Backward-Forward Chaining*** Backward-Forward Chaining is used to first identify the potential root cause of an attack and then reconstruct the overall attack steps. As provenance-based alerting may produce a lot of alerts, we need to prioritize them and identify potential root cause alerts of an attack. This can be done by assigning an alert score during *backward searching*, i.e., incrementing alert scores of each predecessor alert on a path.

[17] We used the SPARQL `CONSTRUCT` syntax to generate new triples and link the detected alert to the respective rule.
[18] https://github.com/SigmaHQ/sigma
[19] https://github.com/SigmaHQ/sigma
[20] https://en.wikipedia.org/wiki/YAML
[21] https://attack.mitre.org/techniques/T1204/003/
[22] Last access: 11/26/2021
[23] https://github.com/SigmaHQ/sigma/wiki/Specification

```
PREFIX : <http://w3id.org/sepses/vocab/krystal#>
PREFIX rule: <http://w3id.org/sepses/vocab/rule#>
SELECT  ?s ?p ?o
  WHERE { ?currentAlert ^:provRel* ?s
          <<?s ?p ?o>> rule:hasDetectedRule ?rule.
  }
```

**Listing 6.** Backward searching expressed as SPARQL query.

```
PREFIX : <http://w3id.org/sepses/vocab/krystal#>
PREFIX rule: <http://w3id.org/sepses/vocab/rule#>
CONSTRUCT { ?s ?p ?o.}
   WHERE { ?startingNode :provRel* ?s . ?s ?p ?o.
       ?s rule:intTag ?spt. ?o rule:intTag ?spo.
  FILTER ( ?spt < 0.5 && ?spo < 0.5 )
}
```

**Listing 7.** Forward chaining mechanism expressed as SPARQL query.

For this search, we leverage *Property Paths*[24], a SPARQL feature that allows us to find routes between nodes in the RDF provenance graph. Recall that we used **rdfs7** inference to automatically generate an upper-level relation `:provRel` between system objects.

As shown in Listing 6, the backward search query consists of a triple pattern [ `?currentAlert` $\wedge$`:provRel`$^*$ `?s` ], in which $\wedge$`.`$^*$ represents the *property path* that finds all possible backward connections from a node `?currentAlert` to a node `?s` via the relation property `:provRel`, where `?s` is classified as part of another alert. We represent this condition as an RDF-star[25] statement, i.e., [ `<<?s ?p ?o>> rule:hasDetectedRule ?rule` ].

Next, we iteratively update the alert scores for each predecessor alert (+1). After scoring all alerts, we can construct attack sequences, starting with the alerts with the highest values and performing *forward chaining* to construct attack scenario graphs. We use the same technique as we did for backward chaining, i.e., property paths to find forward routes connected to the defined starting nodes. Listing 7 shows the generic SPARQL query to construct an attack scenario from the provenance graph. From the `?startingNode`, it finds possible routes and visits connected nodes via the `:provRel` relation. A threshold defines which nodes will be connected, e.g., only low integrity nodes with an integrity tag lower than 0.5.

***Graph Querying*** Graph querying can detect attack behavior in a provenance graph based on attack patterns. The graph query patterns can be constructed from observed behavior or existing information in published CTI, incident reports, public malware documentation, etc. Based on that, the patterns can be constructed manually or – potentially – also through automated extraction methods such as AttacKG (King and Chen, 2003b).

Figure 8 visualizes a graph query example. A subset of the graph (red box) represents an observed attack pattern inside the provenance graph. Unlike previous work (Gao et al., 2018a; 2018b; Milajerdi et al., 2019a) that use custom domain-specific languages to perform graph querying, *KRYSTAL* provides a uniform graph querying mechanism with a high expressivity through SPARQL. Furthermore, it also supports linking to other datasets.

To formulate graph patterns in SPARQL[26], we can define system object types as nodes (i.e., via *Classes*) and their interactions as relation properties (i.e., via *Object Properties*). The formulated graph queries can be executed against the provenance graphs

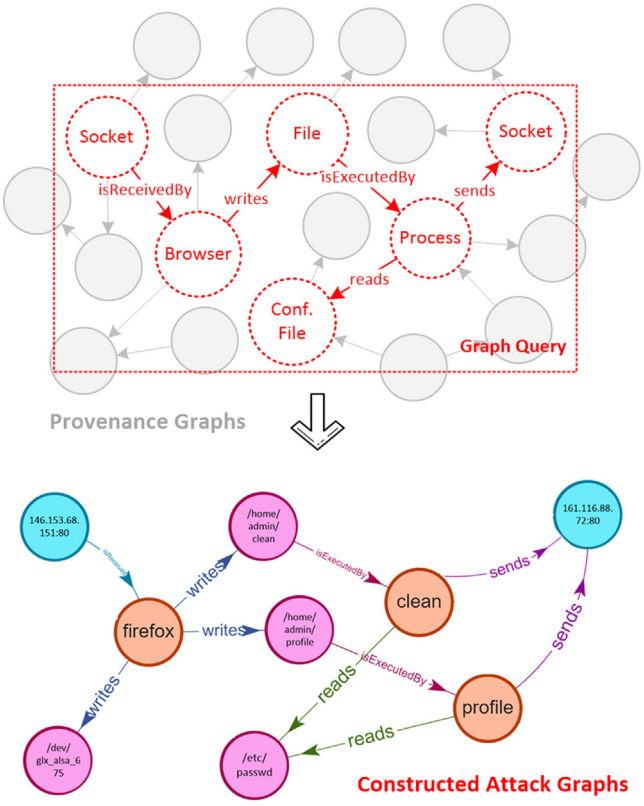


**Fig. 8.** Graph query alignment example: Graph query (red box) is aligned over the provenance graph to detect potential attack patterns. The constructed attack graph below represents a detected pattern. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

```
PREFIX rule: <http://w3id.org/sepses/vocab/rule#>
PREFIX : <http://w3id.org/sepses/vocab/krystal#>

CONSTRUCT { ?socket :isReceivedBy ?browser.
      ... #similar to where clause
 }WHERE {
     ?socket :isReceivedBy ?browser.
    SERVICE
    ↪  <http://w3id.org/sepses/repositories/knowledge>
     {  ?browser a :Browser. }
  ?browser :writes  ?file1. ?file1 :isExecutedBy
  ↪  ?process.
  ?process :reads ?file2.?process :sends ?socket2.
  ?file2 rule:confTag ?fit. FILTER (?fit < 0.5)
}
```

**Listing 8.** SPARQL graph query[26] for Fig. 8.

to match potential attack patterns. Listing 8[27] depicts an example graph query visualized in Fig. 8. It detects a potential attack where a browser receives a connection from a socket (`?socket :isReceivedBy ?browser`). The fact that `?browser` is of type *:Browser* is established in the background knowledge (i.e. `<http://w3id.org/sepses/..knowledge>`). The SPARQL query federation mechanism is used to include the background knowledge in the query (`SERVICE` syntax).

Subsequently, the browser creates a file in the local system (defined by `?browser :writes ?file1`). This file is then exe-

[24] https://www.w3.org/TR/sparql11-query/#propertypaths

[25] https://w3c.github.io/rdf-star/cg-spec/editors_draft.html

[26] Graph pattern that identifies a specific known attack pattern; defined based on the DARPA Transparent Computing TA5.1 Ground Truth Report (DAR, 2021).

[27] Note that the automatic construction of graph patterns from existing resources is out of scope for this paper.

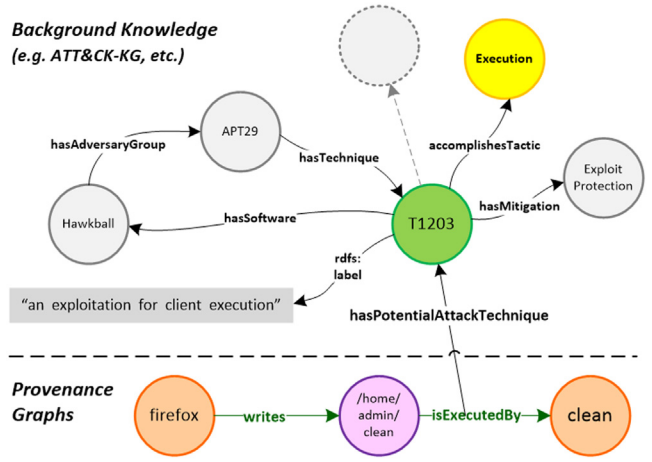


**Fig. 9.** Background linking example, to automatically link alerts detected by the threat detection module to external background knowledge (e.g. ATT&CK-KG).

cuted as a new process (`:file1 :isExecutedBy :process2`) and reads another *confidential* file (`?file1 :reads ?file2`). Next, the process sends the *confidential* file to another socket (`process :sends ?socket2`). We can use the `FILTER` syntax (i.e. `FILTER (?fit < 0.5)`) to focus on confidential files (i.e., with a tag value (`?file2 rule:confTag ?fit`) lower than 0.5).[28]

To reconstruct the resulting attack graph, we used SPARQL `CONSTRUCT` queries to generate new triples in RDF. The generated attack graph can be shared and reused – e.g., for further threat hunting activities and analysis.

### 6.4. Contextualization & linking

Security-related events are typically highly context-specific and hence, their interpretation requires extensive background knowledge (Ekelhart et al., 2018). Such knowledge plays an important role in our approach and can enrich and provide additional information – e.g., to identify high-level attack steps in the generated attack graph.

In particular, we link results to our previously developed SEPSES CSKG (Kiesling et al., 2019), a continuously updated cybersecurity knowledge graph that integrates data from various publicly available sources, including CAPEC, CPE, CVE, CVSS, and CWE. Furthermore, we extend the SEPSES CSKG with attack patterns from MITRE ATT&CK that consist of 665 attack techniques and 14 attack tactics (ATT&CK-KG Kurniawan et al., 2021).

Figure 9 illustrates this background knowledge linking. The example relation `:isExecutedBy` between a file `:/home/admin/clean` and a process `:Clean` can be linked to attack techniques and tactics in the background knowledge: in this case, the technique "*exploitation for client execution*", identified by node `:T1203`) from the MITRE ATT&CK matrix[29] (Kurniawan et al., 2021). Since node `:T1203` also provides links to additional information (tactics, mitigations, etc.), we can use this node to abstract from concrete indicators to a higher-level conceptualization of attacks and their techniques, tactics, procedures.

Listing 9 shows an excerpt of background linking during *forward chaining* via SPARQL query federation. We modified the query from Listing 7 and introduced additional

```
PREFIX : <http://w3id.org/sepses/vocab/krystal#>
PREFIX rule: <http://w3id.org/sepses/vocab/rule#>
PREFIX at: <http://w3id.org/sepses/vocab/ref/attack#>
CONSTRUCT {
  <<?s ?p ?o>> :hasPotentialAttackTechnique ?tech.
  ?s ?p ?o. ?tech at:accomplishesTactic ?tt.}
WHERE {
  OPTIONAL{
   <<?s ?p ?o>> rule:hasDetectedRule ?r.
   SERVICE
   ↪  <http://w3id.org/sepses/repositories/knowledge>
    {?r rule:hasAttackTechnique ?tech.
     ?tech at:accomplishesTactic ?tt.}
  }
   .. #same as forward-chaining query where clause
}
```

**Listing 9.** Contextualization and linking through semantic query federation (SPARQL Service).

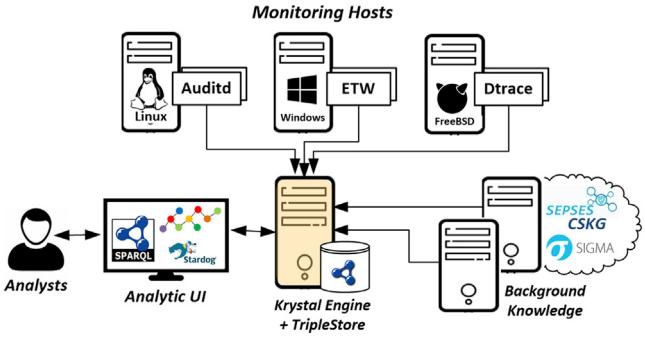


**Fig. 10.** Implementation Setup.

query patterns such as `SERVICE` followed by an endpoint e.g. *http://w3id.org/sepses/repositories/knowledge* which references external background knowledge. This links a detected rule with a corresponding attack technique in the background knowledge (identified by the triple pattern `?r rule:hasAttackTechnique ?tech`). Finally, we also link the identified technique to a tactic (identified by `?tech at:accomplishesTactic ?tt`). This illustrates how the SPARQL federation mechanism makes it possible to query and link an entity to additional resources, such as mitigation techniques, impacts, etc.

## 7. Implementation & application scenarios

In this section, we describe the implementation of our framework and demonstrate its feasibility in several scenarios.

### 7.1. Implementation

Figure 10 visualizes the implementation architecture of our approach. We developed a Java-based log processing tool called the *KRYSTAL* engine[30] that consumes log data from three different log sources, i.e., Linux *auditd*, FreeBSD *DTrace*[31], and Windows *ETW*. We used the *KRYSTAL* Ontology to parse and map log data into an RDF-based provenance graph. By means of the Jena[32] reasoning engine, we can infer new knowledge during provenance graph building.

[28] Recall that semantic reasoning infers the type (i.e. *Class*) of system objects automatically based on their relations (i.e., **rdfs2 & rdfs3**), hence, we can define attack patterns based on their relations only.

[29] https://attack.mitre.org/techniques/T1203/

[30] https://github.com/sepses/SimpleLogProvenance

[31] https://wiki.freebsd.org/DTrace/

[32] https://jena.apache.org/documentation/inference/

**Table 3**
Attack Scenarios.

| Scenario ID | Dataset ID | OS Platform | Scenario Name | Scenario description |
|---|---|---|---|---|
| 1 | Cadets I | FreeBSD | Nginx backdoor | *Nginx backdoor w/ Drakon in-memory*. An attacker sent a malformed HTTP request to a vulnerable Nginx web server that leads to several malicious file creations and process executions in the local system (Figure 11). |
| 2 | Cadets I | FreeBSD | Nginx backdoor | *Nginx backdoor w/ Drakon in-memory*. A vulnerable Nginx webserver downloads several malicious files after being exploited by a malformed HTTP request (Figure 12). |
| 3 | Cadets II | FreeBSD | Nginx backdoor | *Nginx backdoor w/ Drakon in-memory*. Similar to Scenario 3, the vulnerable Nginx webserver was successfully exploited by an attacker. It downloads a payload file which leads to sensitive information leaking (Figure 13). |
| 4 | Theia | Ubuntu 12.04 | Firefox backdoor | *Firefox backdoor w/ Drakon in-memory*. Firefox process gets exploited by a malicious website to download and execute files to steal sensitive information from users (Figure 11). |
| 5 | Five Direction | Windows 10 | Firefox backdoor | *Firefox backdoor w/ Drakon in-memory*. Firefox process gets exploited via a drakor memory payload after browsing a malicious website (Figure 15). |

Furthermore, we extended the existing SEPSES Cybersecurity Knowledge Graph (Kiesling et al., 2019) by including attack techniques and tactics from MITRE ATT&CK and incorporate it as external background knowledge. Furthermore, we translate existing IoC *Sigma* (Roth and Patzke, 2021) rules into SPARQL queries.

To construct attack graphs (through *backward-forward chaining* and *graph querying*), link internal and external background knowledge (via SPARQL query federation), as well as to visualize the resulting attack graphs, we use the in-memory Jena TDB[33] and the *Stardog Enterprise Knowledge Graph platform*.[34]

### 7.2. Application scenarios

In the following application scenarios, we demonstrate how the *KRYSTAL* framework automatically constructs compact attack graphs, links and contextualizes them with background knowledge, and finally maps them to high-level attack steps via TTPs defined in MITRE ATT&CK.

We use a DARPA dataset (DAR, 2021) that contains attack scenarios carried out by a red team as part of the DARPA Transparent Computing (TC) program. An overview of the attack scenarios can be found in Table 3. Due to space limitation, we only explain one scenario in this section (Scenario 1), and include the remaining scenarios in the appendix. *Scenario 1 - Nginx backdoor w/ Drakon in-memory* This attack scenario has been used as motivation example in Section 1. As shown in Fig. 11, our system detected a number of alerts including *file creation, change permission, file execution, and data-leak*. After performing *backward and forward* chaining, we successfully constructed the attack graph of this scenario. Furthermore, we performed query federation during *forward chaining* to include external background knowledge. Our query yields an attack graph that automatically links the detected alerts to the MITRE ATT&CK techniques and tactics. Finally, we can see the reconstructed attack graph together with its kill-chain phases such as *Reconnaissance, Defense Evasion, Execution* and *Exfiltration*.

## 8. Evaluation

In this section, we evaluate the *KRYSTAL* framework through a set of experiments and discuss the results.

### 8.1. Experimental setup

We performed the experiments on two machines: (i) an Ubuntu 18.04 Server (Intel 2.59 GHz vCPU, 32 GB RAM) was used for provenance graph building and evaluation of threat detection and alerting techniques. (ii) a Windows 10 (Intel 2.90 GHz vCPU, 16 GB RAM) machine was used for scenario reconstruction, graph querying and attack graph visualization using *Stardog Studio*.[35]

***Dataset Overview*** For the evaluation, we used well-established datasets from red vs. blue team adversarial engagements produced as part of the third Transparent Computing (TC) program organized by DARPA (DAR, 2021). The datasets are organized into five categories, namely *Cadets, Trace, Theia, FiveDirections and ClearScope*. Each dataset includes log events generated during the engagements on a specific targeted host and platform. For example, *Cadets* represents *dtrace*[36] log data from FreeBSD OS, *Trace* and *Theia* have been generated from *auditd*[37] Ubuntu log data, and *FiveDirection* contains *ETW*[38] log data from Microsoft Windows and *ClearScope* collected from Android logs. In addition, a description of attack steps is available as ground truth. Table 4 summarizes the five attack scenarios from the *Theia* (TH), *Cadets* (CD), and *FiveDirection* (FD) datasets that we evaluated. In total, the scenarios covers more than 7 days of log data from three datasets, with more than 53 GB of logs in JSON format.[39]

### 8.2. Experiment results

We collect the following evaluations metrics in our experiments: (i) provenance graph size reduction and compression performance, (ii) run-time performance, (iii) provenance-based alert detection, (iv) rule-based alert detection.

*Graph Size Reduction & Compression Performance* Table 4 shows the evaluation results for provenance graph generation and compression in the five scenarios.

Our system generates RDF-based provenance graphs in RDF TURTLE format.[40] Out of the 18.7 GB Theia log dataset, for instance, we generated a 280 MB provenance graph (i.e., 66x smaller), the set of Cadets datasets (19 GB) resulted in a 400 MB provenance graph (i.e., 48x smaller), and out of the Five direction dataset (16 GB), we generated a 25 MB provenance graph (i.e., 640x smaller).

The last column of Table 4 shows the compression results of the generated provenance graphs for each dataset in HDT (Fernández et al., 2013) format. On average, the resulting compressed provenance graphs are approximately smaller by a factor of 22 than the generated provenance graphs in TURTLE format.

***Run-time Performance*** We evaluate three aspects to measure the run-time performance of our approach: (i) Time for generating the provenance graphs from the log data, including time for *tag-propagation, attenuation & decay* and *provenance-based alerting*; (ii)

[33] https://jena.apache.org/documentation/tdb/
[34] https://www.stardog.com/platform/
[35] https://www.stardog.com/studio/
[36] https://en.wikipedia.org/wiki/DTrace
[37] https://linux.die.net/man/8/auditd
[38] https://docs.microsoft.com/en-us/windows/win32/etw/event-tracing-portal
[39] Note that DARPA published the datasets in both binary and JSON formats, we used the JSON data as input in our evaluation.
[40] https://www.w3.org/TR/turtle/

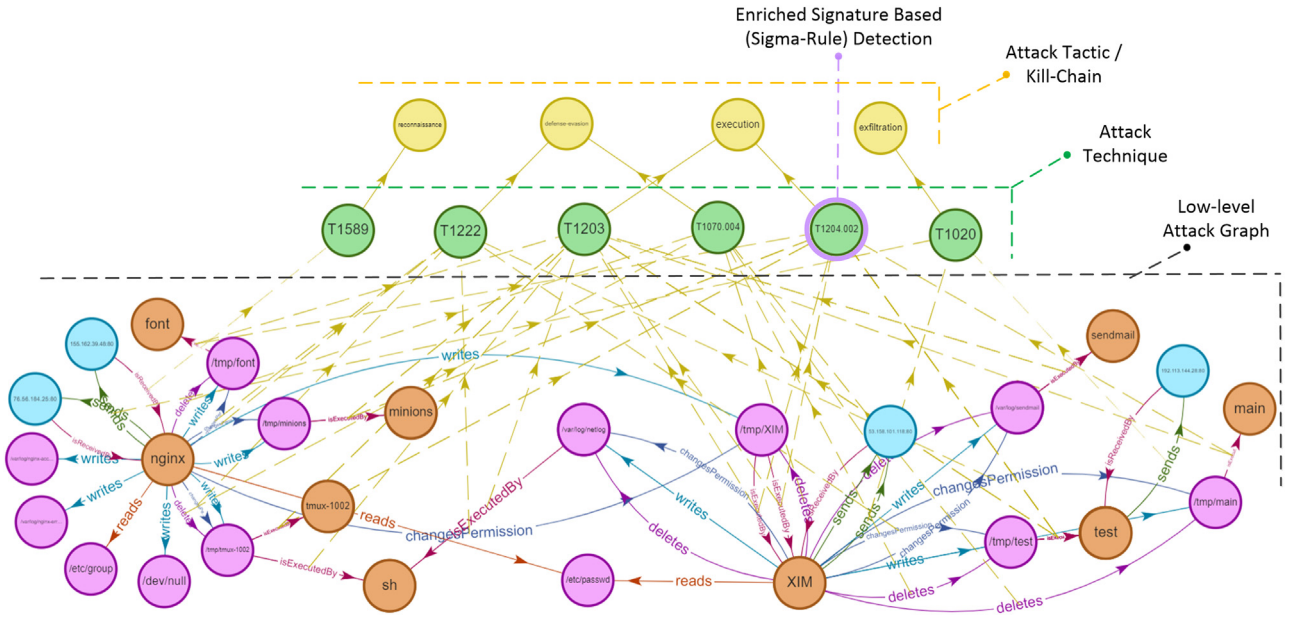


**Fig. 11. Scenario 1** (*Nginx backdoor w/ Drakon in-memory*). This attack begins with a vulnerable Nginx web server hosted on a FreeBSD server that gets exploited by a malformed HTTP request. The exploit leads to multiple file creations on the local system. The attacker successfully creates an executable file ("/tmp/XIM"), changes the permissions and runs it as an elevated process. This process reads a sensitive file ("/etc/passwd") and forwards data to an external network (*53.158.101.118:80*).

**Table 4**
Graph size reduction & compression.

| Scenario ID | Dataset ID | Duration (hh:mm) | Log data in JSON (GB) | Prov. in RDF (MB) | Prov. in HDT (MB) |
|---|---|---|---|---|---|
| 1,2 | CD I | 48:59 | 7 | 150 | 7 |
| 3 | CD II | 90:01 | 12 | 250 | 10 |
| 4 | TH | 25:33 | 18.7 | 280 | 16 |
| 5 | FD | 19:27 | 16 | 25 | 0.9 |

**Table 5**
Scenario graph construction run-time.

| Scenario ID | Total Events (M) | Prov. Graph Building & Alerting (events/sec) | Forward Chaining (sec) | Graph Querying (sec) |
|---|---|---|---|---|
| 1,2 | 7.8 | 16.8 K | 0.79 | 0.31 |
| 3 | 12.9 | 17.5 K | 0.99 | 0.26 |
| 4 | 23.4 | 15.6 K | 1.6 | 0.41 |
| 5 | 19.3 | 37.8 K | 1.99 | 0.42 |

Time for constructing the scenario graphs via *list:forward-chaining*; and (iii) Time for generating scenario graphs through *graph querying*.

We repeated each experiment five times and present average results.

As shown in Table 5, our approach can generate RDF-based provenance graphs from log data with up to 20k events/sec on average. The highest performance for provenance generation is achieved in Scenario 5 (*FiveDirection* dataset on Windows), with 37.8k events/sec. Compared to the other scenarios, *FiveDirection* has a larger number of events that are not considered in our model, resulting in a large number of events that can be excluded in the provenance graph generation process.

Compared to MORSE, which achieved 100K events/sec (Hossain et al., 2020), provenance construction is somewhat slower.[41] This performance penalty with respect to approaches based on optimized custom data structures is expected and mainly attributable to the parsing, RDF lifting and the in-memory SPARQL query execution times necessary for tag propagation. The manageable reduction in run time performance in our evaluation demonstrates that the approach is viable. Given the benefits, including improved reusability, interoperability, and enrichment with background knowledge the tradeoff seems favorable. Overall, our framework achieved a performance of 1.2 s per attack on average in the scenario graph construction via *list:forward-chaining*. The highest performance has been achieved in scenario 2 and 3 (*Cadets* dataset with FreeBSD) with 0.79 s. Note that we excluded time for *backward-chaining* for root cause identification as it is basically a select query over the provenance graph (without constructing a new graph) and therefore the run times are relatively fast. The run time for scenario graph generation through *graph querying* is even faster, i.e., less than 0.5 s for all scenarios. This indicates that our RDF-based provenance graph data model scales well with respect to graph size and query complexity when it comes to graph-query based attack construction.

***Propagation-Based Alert Detection Performance*** In the following, we measure the effectiveness of our approach in detecting alerts based on alerting policies over the tagged provenance graph. We leverage alert policies such as *Change Permission*, *File Execution*, *File Corruption*, or *Data Leak* from (Hossain et al., 2020). In addition, we created a custom alert *Reconn* that detects connections from external IPs to processes that access sensitive files. Table 6 summarizes the detected alerts for all scenarios. Overall, our approach detected all high-level attack activities in the ground truth and achieved similar detection performance as the evaluation in Hossain et al. (2020) (cf. Table 3).

[41] Specifically, by a factor of 5 for Linux/FreeBSD and 3 for Windows, respectively.

**Table 6**
Provenance-based alert detection.

| Scenario ID | Total Events (M) | Reconn | Change Perm | File Exec | File Corrupt | Data Leak |
|---|---|---|---|---|---|---|
| 1,2 | 7.8 | 65 | 1152 | 15 | 115 | 3 |
| 3 | 12.9 | 4 | 1274 | 1 | 1040 | 3 |
| 4 | 23.4 | 3 | 9 | 2 | 618 | 3 |
| 5 | 19.3 | 22 | 448 | 0 | 288 | 10 |

**Table 7**
Detected alerts by *Sigma* rules.

| Scenario ID | Dataset ID | Total Events (M) | Correctly Identified Alert | Incorrectly Identified Alert |
|---|---|---|---|---|
| 1,2 | CD I | 7.8 | 34 | 0 |
| 3 | CD II | 12.9 | 41 | 0 |
| 4 | TH | 23.4 | 19 | 0 |
| 5 | FD | 19.3 | 1162 | 261 |

***Signature-Based Alert Detection Performance*** In this evaluation, we used the *Sigma* rules incorporated into our *KRYSTAL* framework to detect attacks based on IoCs from logs. As discussed in Section 6.2, we automatically translated *Sigma* rules into executable SPARQL queries and run them against the provenance graph. To this end, we translated most of the existing *Sigma* rules for Linux logs (33 rules) and Windows logs (160 rules). At the time of writing[42], there are no specific *Sigma* rules for *dtrace* FreeBSD, however, *dtrace* FreeBSD logs have a similar structure to *auditd* Linux logs in the evaluated DARPA dataset, hence, we could also use them to detect attacks in FreeBSD logs.

Table 7 shows the number of triggered alerts based on *Sigma* rules from all five scenarios within our *KRYSTAL* framework. For scenarios with Linux and FreeBSD as log sources, the translated *Sigma* rules detected similar alerts as *propagation-based alert detection* without any incorrectly identified alerts. It includes alerts for *system owner or user discovery*[43], *file or folder permissions change*[44], *privilege escalation preparation*[45], *program executions in suspicious folders*[46], *etc*. Furthermore, *Sigma* rules detected more specific alerts which have been missed by *propagation-based approaches*, such as *bash_profile modification*[47] (as part of the *persistence* phase), and *data compressed*[48] (as part of the *data-exfiltration* phase).

For Windows, we identified more alerts on Scenario 5 (Five Direction) with a total of 1162 alerts (with 261 incorrect alerts that could not be linked to actual attack activities in the scenario). Relevant alerts include, e.g., *Suspicious Service Path Modification*[49], *Suspicious XOR Encoded PowerShell Command Line*[50], *Rar with Password or Compression Level*[51], *Change Default File Association*[52], *LSASS Memory Dumping*[53], and *Capture a Network Trace with netsh.exe*.[54]

***Integrated and Enriched Cross-Technique Graphs***

Our experiments showed that the knowledge graph foundation enables the integration of results from various detection techniques and their linking to additional knowledge and gives the analyst a rich view for comprehensive, multi-paradigmatic threat analysis.

The scenario attack graph in Fig. 11, for instance, shows an enriched attack graph constructed through a combination of techniques summarized in Table 8, which compares *KRYSTAL* to other state of the art approaches and highlights the integrative aspect of the framework. The identified attack steps are linked to rich background knowledge on techniques, tactics, procedures (SEPSES ATT&CK-KGKurniawan et al. (2021)) in (cf. Fig. 9, detailed information on tactics and techniques not shown due to space constraints).

## 9. Discussion

In this section, we discuss how *KRYSTAL* can support threat detection and attack reconstruction processes and cover its limitations.

***Uniform Data Model and Representation*** A uniform representation is a key foundation to be able to fulfill the requirements put forth in Section 4. Existing provenance-graph based detection and investigation approaches lack a unified data format, which hinders their reuse and integration. *KRYSTAL* fills this gap (also cf. Li et al., 2021) with a knowledge graph framework based on the W3C standards RDF and OWL.[55] The unified model allowed us to combine various state of the art threat detection techniques and apply them on a common provenance graph – fulfilling R2. Furthermore, it also also makes it easier to construct datasets and share provenance graph data.

Our evaluation showed that the *KRYSTAL* ontology can be used to model audit data across platforms – i.e., Linux (*auditd*), FreeBSD (*dtrace*), and Windows (*ETW*) – thereby fulfilling R4. Overall, this should contribute towards lowering the barrier for further research, reproduction, and quantitative comparison.

Finally, we also find that the uniform representation makes it possible to contextualize provenance graphs with knowledge from internal and external sources (R1). This is particularly useful for *KRYSTAL*'s ability to not only reconstruct *low-level* attack graphs, but also link them to *high-level* attack tactics and techniques from MITRE ATT&CK (R3).

Future approaches building on our model can take advantage of the semantic flexibility and richness offered by the existing RDF ecosystem. RDF can, for instance, support multiple paradigms for the implementation of data management architectures in provenance graph-based detection systems, including (i) materialized graphs in triple stores, (ii) cached graphs implemented as in-memory triple stores (Hu et al., 2016), (iii) distributed graphs (Abdelaziz et al., 2017; Gu et al., 2014; Lehmann et al., 2017), (iv)

[42] last access 04/05/2021
[43] http://bit.ly/sigmaDiscovery
[44] http://bit.ly/sigmaFolderPermission
[45] http://bit.ly/sigmaPrivilegeEscalation
[46] http://bit.ly/sigmaProgramExecution
[47] http://bit.ly/sigmaBashProfileModification
[48] http://bit.ly/sigmaDataCompressed
[49] http://bit.ly/SigmaWinSuspService
[50] http://bit.ly/SigmaWinPowerShellXOR
[51] http://bit.ly/SigmaWinRarFlags
[52] http://bit.ly/SigmaWinChangeFileAssoc
[53] http://bit.ly/SigmaWinLSASSDump
[54] http://bit.ly/SigmaWinNetSHPacket

[55] cf. https://www.w3.org/standards/

**Table 8**
Detection techniques supported by state of the art approaches.

| Detection Technique | Holmes (Milajerdi et al., 2019b) | Morse (Hossain et al., 2020) | Poirot (Milajerdi et al., 2019a) | Rapsheet (Hassan et al., 2020) | Krystal |
|---|---|---|---|---|---|
| Tag-Propagation | - | ✓ | - | - | ✓ |
| Rule/Policy-Based | ✓ | ✓ | - | - | ✓ |
| Signature-Based | - | - | - | ✓ | ✓ |
| Graph Query | - | - | ✓ | - | ✓ |
| Tactical Analysis (TTP Mapping) | ✓ | - | - | ✓ | ✓ |

virtualized graphs (Kurniawan et al., 2022; Xiao et al., 2019), and (v) stream reasoning techniques Dell'Aglio et al. (2017).[56]

***Standard Query Language*** *KRYSTAL* leverages SPARQL Consortium et al. (2013), a graph-based query language for RDF that offers high expressivity and supports complex querying (e.g., aggregation, subqueries, negation) in a declarative manner Kaminski et al. (2016). We find that this standardized query language provides powerful means to define reusable rules, policies and graph patterns (cf. R2).

In particular, we observe that SPARQL *property path* queries can perform analyses that are critical to provenance-based analyses – such as *backward-forward* chaining for attack graph reconstruction – efficiently. This is despite the fact that theoretical studies Arenas et al. (2012) on the computational complexity of property paths found that the implementation of a naïve (unfiltered) query can result in double exponential runtime complexity, which becomes critical, e.g., for nodes with more than 15k sequence paths. The typical property path lengths in our scenario attack graphs, however, tend to be rather short, with a maximum path length of 37. This is attributable to the absence of long chains in the provenance data to begin with on the one hand, but also due to tagging and filtering mechanisms such as attenuation and decay, which effectively limit path lengths by focusing only on relevant suspicious events and nodes with low integrity. As a consequence, we observe that the attack graph reconstruction through property paths performs efficiently.

Although SPARQL is part of computer science curricula and is increasingly being adopted in many industries[57], it is typically new to security analysts and thus requires some training. This could partly be addressed with general-purpose visual query building and exploration tools such as Haag et al. (2014), Vargas et al. (2019), but we also see a potential for future work in the development of intuitive specialized interfaces for threat detection and attack reconstruction.

***Integrated & Modular Framework*** Current research has resulted in numerous prototypes that each provide solutions for specific detection techniques. *KRYSTAL*, by contrast, enables the combination of multiple different threat detection techniques and attack reconstruction approaches in a single integrated framework (cf. R2). Instead of applying different techniques – each with their own preprocessing pipelines – in isolation, the framework allows us to compare and combine different techniques in a single model. In Sections 6 and 7, we specifically showed how a variety of detection and attack reconstruction techniques can be formulated in SPARQL and executed within the *KRYSTAL* framework. The modularity of the framework also makes it extensible for future techniques.

***Distributed Log Analysis*** In production settings, security analysis will often require and involve data from disparate sources. *KRYSTAL* is built upon Semantic Web technologies which are explicitly designed for decentralization. Consequently, *KRYSTAL* inherently supports distributed analysis (i.e., querying data across different machines). In Sections 6 and 7, we demonstrated how *KRYSTAL* can integrate external information sources and facilitate distributed analysis through SPARQL query federation, a technique that enables multiple data sources to be queried in an integrated manner.

To support large-scale provenance graph based attack discovery in production environments, it is necessary to distribute and parallelize computational loads to multiple (local) log processing nodes. This will be part of our future research, where we plan to allow for independent local analysis modules for threat detection and alerting and to integrate the local results into a (global) module, e.g., via SPARQL query federation in order to construct complete attack graphs to address cross-machine attack scenarios. If necessary, several layers of hierarchies can be introduced to better scale the coordination effort.

***Online Attack Graph Reconstruction*** *KRYSTAL* has been evaluated in an offline setting, which can facilitate, e.g., forensic analyses. A (near real-time) online deployment mode would require consideration of issues for attack reconstruction over streaming data. In particular, it would require (i) strategies to dynamically construct attack graphs, (ii) mechanisms to manage continuous updates on a multitude of parallel attack graph reconstruction processes, (iii) policies for prioritizing and discarding attack graph reconstruction processes. Furthermore, the complexity of the approach grows if analyses should be performed in large distributed scenarios and over multiple data streams, which raises issues around time synchronization, latency, throughput, etc. We plan to investigate online scenarios with (semantic) streaming technologies and extend the framework accordingly in future work.

***Unknown Attack Behavior*** To increase robustness against new ways to evade detection, *KRYSTAL* provides a more general framework that makes it possible to combine a variety of attack reconstruction techniques on top of a common knowledge graph foundation. For instance, we demonstrated how *KRYSTAL* can incorporate existing community-based threat detection rules such as Sigma and integrate them with state of the art detection techniques that have been demonstrated to be effective in the context of Advanced Persistent Threat.

We argue that although evasion techniques to circumvent detection will always be a concern, the possibility to apply multiple techniques (combining, e.g., rule-based, graph queries, and tag propagation) in the KRYSTAL framework can provide more robust detection of unknown attack behavior compared to the isolated application of individual approaches. Furthermore the flexible framework allows for faster adaptation, experimentation, and parameter tuning. For instance, the rules and policies in the Listings in this paper can be adapted quickly to address new evasion techniques.

## 10. Conclusions

In this paper, we proposed *KRYSTAL*, a modular knowledge graph-based framework for threat detection, scenario reconstruction, and tactical attack analysis. We provide an open, standards-based provenance graph representation based on Semantic Web technologies that can flexibly combine multiple threat detection

[56] For a survey of RDF data storage and query processing schemes, cf. Wylot et al. (2018). For a survey of approaches to scale to massive data, cf. Ma et al. (2016).

[57] cf. http://sparql.club

techniques and contextualize provenance data over both internal system knowledge and external cyber-security knowledge. Based on the *KRYSTAL* ontology, we provide a foundation for provenance and attack graph modeling in a unified framework. The ontology provides semantic interoperability and allows users to leverage community knowledge for tactical attack analysis. Furthermore, our framework introduces a declarative and modular architecture to overcome the inflexibility of monolithic prototypes; hence, it supports rapid development and integration of new approaches, lowering the barriers for rule development, reproducibility, and further research.

To evaluate the ability of SPARQL to effectively express threat detection rules, we implemented several state of the art techniques including *tag propagation, list:attenuation* and *decay, signature/rule-based* detection and *graph queries*. We evaluated the feasibility of our approach for threat detection and attack construction through multiple attack scenarios from the well-established DARPA-TC dataset. The evaluation shows that our ontology-based RDF provenance graphs are scalable with respect to graph size and query complexity without sacrificing graph reduction, compression, and attack reconstruction performance. This makes it possible to combine a variety of threat detection techniques, which improved the detection capabilities in our evaluation. For instance, we found that complementary rule-based threat detection identified threats which were missed by tag-propagation techniques.

Finally, our framework facilitates linking to high-level attack patterns to establish a "kill-chain" of high-level attacker tactics, which results in a navigable and queryable provenance graph enriched with security knowledge. This can help improve attack understanding and situational awareness.

As next steps in our research, we plan to integrate with multiple large-scale heterogeneous log sources beyond audit log data and aim for performance optimization (e.g., for better exploration and visualization). Based on the possibility to combine detection techniques, we want to explore how alerts raised by one technique could automatically trigger analysis by other techniques for confirmation and additional information. Further research will also focus on exploring other threat detection approaches, including *anomaly detection* techniques, and integrating them into our framework. Finally, we plan to adapt and evaluate the attack graph construction approach in different implementation settings such as in *decentralized* and *distributed* scenarios.

## Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## CRediT authorship contribution statement

**Kabul Kurniawan:** Conceptualization, Methodology, Software, Investigation, Validation, Visualization, Writing – original draft. **Andreas Ekelhart:** Conceptualization, Writing – review & editing. **Elmar Kiesling:** Conceptualization, Writing – review & editing. **Gerald Quirchmayr:** Supervision. **A Min Tjoa:** Supervision.

## Acknowledgments

This work has been supported by netidee SCIENCE and the Austrian Science Fund (FWF) under grant P30437-N31. The competence center SBA Research (SBA-K1) is funded within the framework of COMET — Competence Centers for Excellent Technologies by BMVIT, BMDW, and the federal state of Vienna, managed by the FFG. Moreover, the financial support by the Christian Doppler Research Association, the Austrian Federal Ministry for Digital and Economic Affairs and the National Foundation for Research, Technology and Development is gratefully acknowledged (Christian-Doppler-Laboratory for Security and Quality Improvement in the Production System Lifecycle).

## Appendix

*Scenario 2 - Nginx backdoor w/ Drakon in-memory* In this scenario our system detects several alarms on the provenance graph as shown on Fig. 12. We performed *backward-forward* chaining with query federation to construct the attack graph together with its kill-chain phases. We successfully linked the generated attack graph to MITRE ATT&CK techniques such as *Gather Victim Identity Information* (T1589), *File and Directory Permissions Modification*

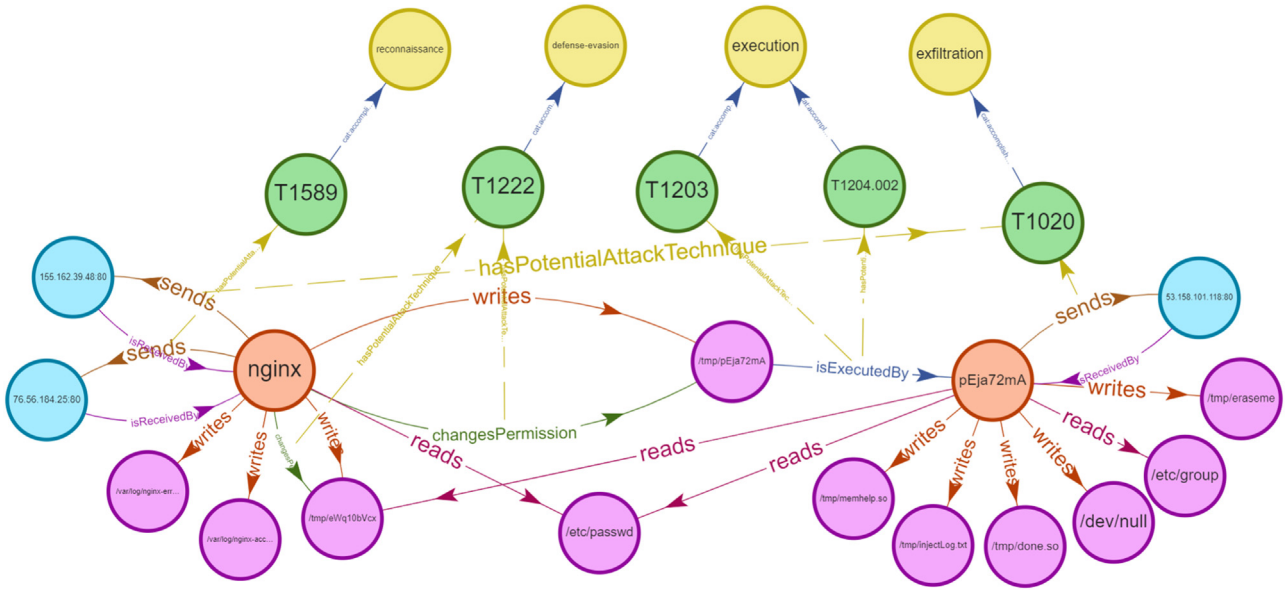


**Fig. 12. Scenario 2** (*Nginx backdoor w/ Drakon in-memory*). The attack begins with a vulnerable Nginx installed on a FreeBSD host that gets exploited by an attacker. The attacker sends a malformed HTTP request that results in downloading several malicious files on the local system. One of the files i.e. */tmp/pEja72mA* then gets executed, which spawns a process *pEja72mA*. This process reads sensitive information*/etc/passwd)* and connects remotely via C&C to the attacker console.

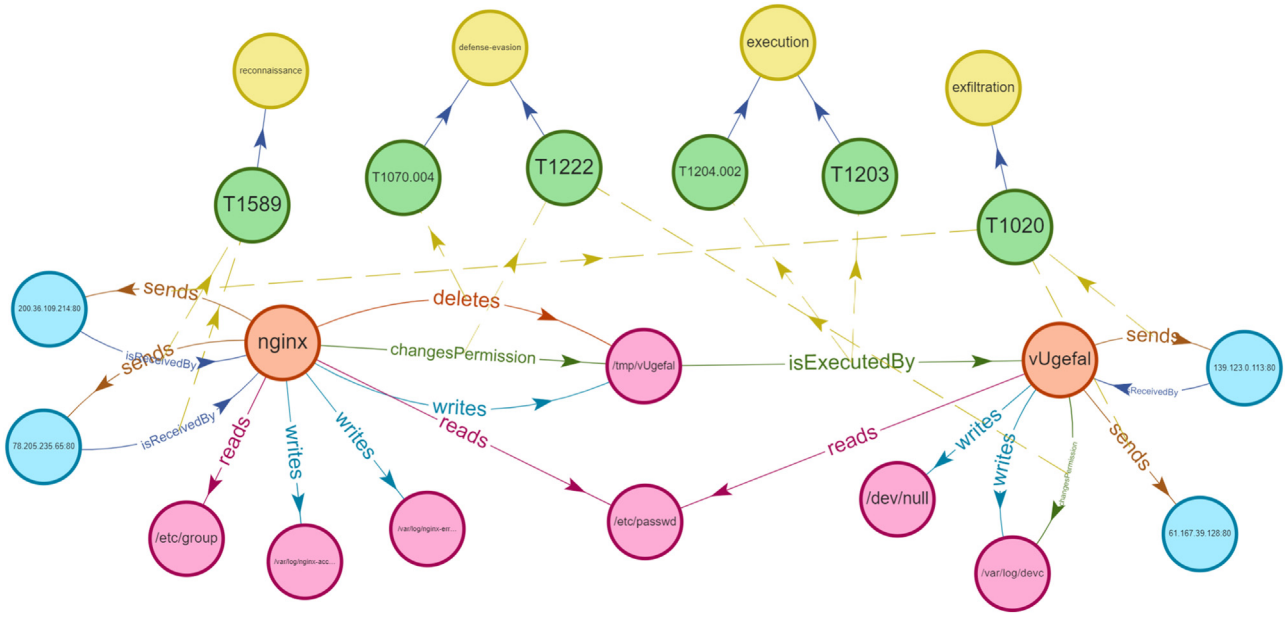


**Fig. 13. Scenario 3** (*Nginx backdoor w/ Drakon in-memory).* The same case as before, this time the attacker successfully exploits a vulnerable Nginx by downloading another payload file ("/tmp/vUgefal"). The attacker spawns an elevated process. This process later reads sensitive information ("/etc/passwd") from the local system and sends data the the external network.

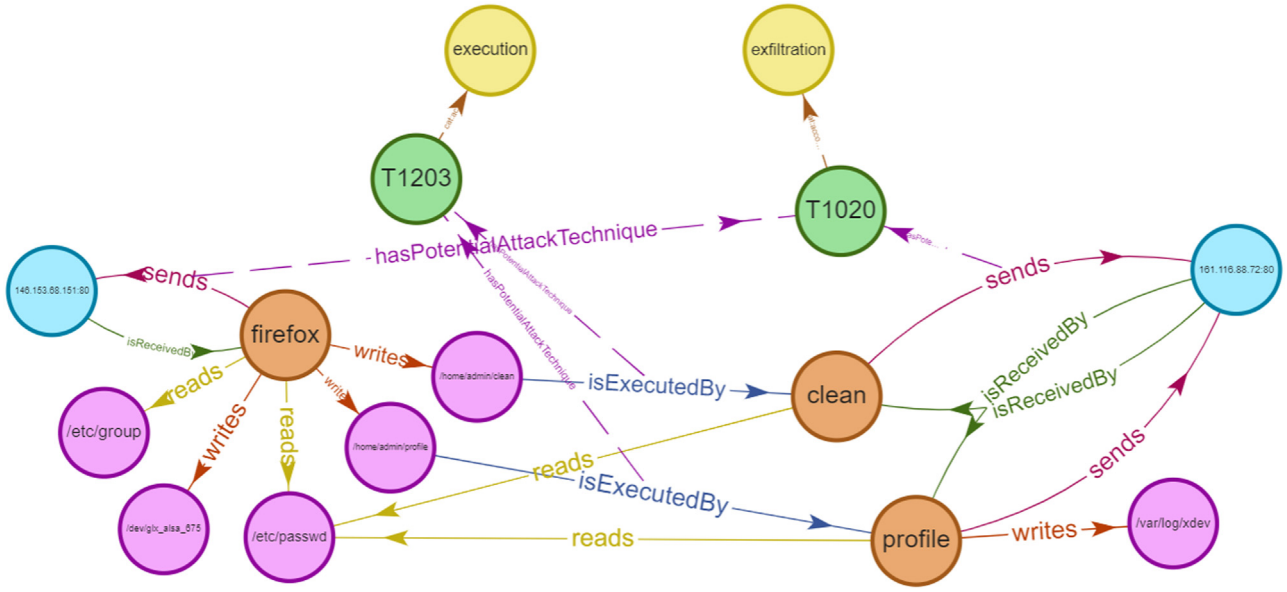


**Fig. 14. Scenario 4** (*Firefox backdoor w/ Drakon in-memory*) The attack starts with the exploitation of Firefox 54.01 on Ubuntu 12.04 by a malicious ad server. The *Firefox* process gets compromised after visiting a malicious website, and subsequently downloads a malicious file to the user directory "*/home/admin/clean*". This file is then executed spawning a new process *Clean*. Another file *Profile* is also created and spawned. Both processes access a sensitive file "*/etc/passwd*" and send the data to an external network (*161.116.88.72:80*).

(T1222), *Exploitationfor Client Execution* (T1203) and *Automated Exfiltration* (T1020). Finally, following the connection from techniques to tactics, we can see kill-chain phases including *Reconnaissance, Defense-evasion, Execution* and *Exfiltration*.

*Scenario 3 - Nginx backdoor w/ Drakon in-memory*

As shown in Fig. 13, from the detected alerts, we performed *backward-forward* chaining over the provenance graphs together with query federation to external background knowledge. The system successfully constructed this scenario attack graph together with connections to MITRE ATT&CK techniques and tactics. In that figure, we see identified attack techniques such as *Gather Victim Identity Information* (T1589), *File and Directory Permissions Modification* (T1222), *Exploitation for Client Execution* (T1203) and *Automated Exfiltration* (T1020). Finally, these detected techniques lead to the attack phases including *Reconnaissance, Defense-evasion, Execution* and *Exfiltration*.

*Scenario 4 - Firefox backdoor w/ Drakon in-memory* As shown in Fig. 14, our system detects several alarms in this scenario, including file execution ("*Clean*" and "*Profile*" process) and data leaks (both "*Clean*" and "*Profile*" processes read a sensitive file "*/etc/passwd*" and send data to "*161.116.88.72:80*"). Furthermore, due to the query federation those detected alarms are automatically linked and mapped to our CTI background knowledge, yielding two MITRE ATT&CK techniques namely *Exploitation for Client*

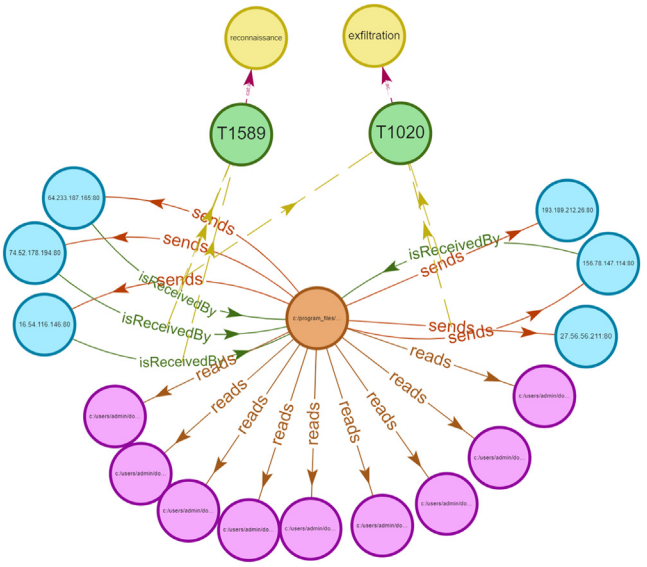


**Fig. 15. Scenario 5** (*Firefox backdoor w/ Drakon in-memory*). This attack begins with the exploitation of Firefox 54.01, installed on a Windows 10 host. The exploit loads drakon into Firefox memory, which opens a C&C connection to the attack network. Via the Firefox process, multiple user documents are successfully exfiltrated to the attacker's network.

*Execution* (T1203) and *Automated Exfiltration* (T1020). Those techniques are further linked to high-level MITRE ATT&CK tactics, i.e. *Execution* and *Exfiltration*.

*Scenario 5 - Firefox backdoor w/ Drakon in-memory* As shown in Fig. 15, our system detects several alerts within this scenario. After performing *backward-forward* chaining and queries with federation to background knowledge, our system successfully identified two MITRE ATT&CK attack techniques within the attack graph, namely *Gather Victim Identity Information* (T1589) and *Automated Exfiltration* (T1020). Finally, we identify kill-chain phases such us *Reconnaissance* and *Exfiltration*.

## References


Abdelaziz, I., Harbi, R., Khayyat, Z., Kalnis, P., 2017. A survey and experimental comparison of distributed SPARQL engines for very large RDF data. Proc. VLDB Endowment 10 (13), 2049–2060. https://doi.org/10.14778/3151106.3151109

Arenas, M., Conca, S., Pérez, J., 2012. Counting beyond a Yottabyte, or how SPARQL 1.1 property paths will prevent adoption of the standard. In: WWW '12: Proceedings of the 21st International Conference on World Wide Web. Association for Computing Machinery, New York, NY, USA, pp. 629–638. doi:10.1145/2187836.2187922.

Capec, 2021Common attack pattern enumeration and classification. Accessed: 30.03.2021, https://capec.mitre.org/.

Consortium W.W.W., et al. SPARQL 1.1 overview. 2013. https://www.w3.org/TR/sparql11-overview/.

CVE, 2021Common vulnerabilities and exposures. Accessed: 30.03.2021, https://cve.mitre.org/about/index.html.

CVSS, 2021Common event expression. Accessed: 30.03.2021, https://cee.mitre.org/.

CVSS, 2021Common vulnerability scoring system. Accessed: 30.03.2021, https://www.first.org/cvss/.

CWE, 2021Common weakness enumeration. Accessed: 30.03.2021, https://cwe.mitre.org/about/index.html.

Cybox, 2021Cyber observable expression. Accessed: 30.03.2021, https://cyboxproject.github.io/.

DARPA, 2021Transparent computing engagement 3 data release. Accessed: 02.02.2021, https://drive.google.com/drive/folders/1QlbUFWAGq3Hpl8wVdzOdIoZLFxkII4EK.

Dell'Aglio, D., Della Valle, E., van Harmelen, F., Bernstein, A., 2017. Stream reasoning: a survey and outlook. Data Sci. 1 (1–2), 59–83. https://doi.org/10.3233/DS-170006.

Ekelhart, A., Ekaputra, F.J., Kiesling, E., et al., 2021. The SLOGERT framework for automated log knowledge graph construction. In: The Semantic Web: ESWC 2021. Springer International Publishing, p. 16. https://doi.org/10.1007/978-3-030-77385-4_38

Ekelhart, A., Kiesling, E., Kurniawan, K., et al., 2018. Taming the logs - Vocabularies for semantic security analysis, 137, pp. 109–119. doi:10.1016/j.procs.2018.09.011.

Fernández, J.D., Martínez-Prieto, M.A., Gutiérrez, C., Polleres, A., Arias, M., 2013. Binary RDF representation for publication and exchange (HDT). J. Web Semant. 19, 22–41. http://www.websemanticsjournal.org/index.php/ps/article/view/328.

Gao, P., Xiao, X., Li, D., Li, Z., Jee, K., Wu, Z., Kim, C.H., Kulkarni, S.R., Mittal, P., 2018a. SAQL: a stream-based query system for real-time abnormal system behavior detection. In: 27th USENIX Security Symposium (USENIX Security 18). USENIX Association, Baltimore, MD, pp. 639–656. https://www.usenix.org/conference/usenixsecurity18/presentation/gao-peng

Gao, P., Xiao, X., Li, Z., Jee, K., Xu, F., Kulkarni, S.R., Mittal, P., 2018b. AIQL: enabling efficient attack investigation from system monitoring data. In: USENIX ATC '18: Proceedings of the 2018 USENIX Conference on Usenix Annual Technical Conference. USENIX Association, USA, pp. 113–125. doi:10.5555/3277355.3277367.

Gu, R., Hu, W., Huang, Y., 2014. Rainbow: a distributed and hierarchical RDF triple store with dynamic scalability. In: 2014 IEEE International Conference on Big Data (Big Data). IEEE, pp. 561–566. https://doi.org/10.1109/BigData.2014.7004274.

Haag, F., Lohmann, S., Bold, S., Ertl, T., 2014. Visual SPARQL querying based on extended filter/flow graphs. In: AVI '14: Proceedings of the 2014 International Working Conference on Advanced Visual Interfaces. Association for Computing Machinery, New York, NY, USA, pp. 305–312. doi:10.1145/2598153.2598185.

Hassan, W.U., Bates, A., Marino, D., 2020. Tactical provenance analysis for endpoint detection and response systems. In: 2020 IEEE Symposium on Security and Privacy (SP), pp. 1172–1189. doi:10.1109/SP40000.2020.00096.

Horrocks, I., 2005. OWL: a description logic based ontology language. In: van Beek, P. (Ed.), Principles and Practice of Constraint Programming - CP 2005. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 5–8. https://doi.org/10.1007/11564751_2

Hossain, M.N., Milajerdi, S.M., Wang, J., Eshete, B., Gjomemo, R., Sekar, R., Stoller, S.D., Venkatakrishnan, V.N., 2017. SLEUTH: real-time attack scenario reconstruction from COTS audit data. In: SEC'17: Proceedings of the 26th USENIX Conference on Security Symposium. USENIX Association, USA, pp. 487–504. doi:10.5555/3241189.3241228.

Hossain, M.N., Sheikhi, S., Sekar, R., et al., 2020. Combating dependence explosion in forensic analysis using alternative tag propagation semantics. In: 2020 IEEE Symposium on Security and Privacy (SP). IEEE, San Francisco, CA, USA, pp. 1139–1155. doi:10.1109/SP40000.2020.00064.

Hossain, M.N., Wang, J., Sekar, R., Stoller, S.D., 2018. Dependence-preserving data compaction for scalable forensic analysis. In: SEC'18: Proceedings of the 27th USENIX Conference on Security Symposium. USENIX Association, USA, pp. 1723–1740. doi:10.5555/3277203.3277331.

Hu, C., Wang, X., Yang, R., Wo, T., 2016. ScalaRDF: a distributed, elastic and scalable in-memory RDF triple store. In: 2016 IEEE 22nd International Conference on Parallel and Distributed Systems (ICPADS). IEEE, pp. 593–601. https://doi.org/10.1109/ICPADS.2016.0084.

Ji, Y., Lee, S., Downing, E., Wang, W., Fazzini, M., Kim, T., Orso, A., Lee, W., et al., 2017. RAIN: refinable attack investigation with on-demand inter-process information flow tracking. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. ACM, Dallas Texas USA, pp. 377–390. doi:10.1145/3133956.3134045.

Ji, Y., Lee, S., Fazzini, M., Allen, J., Downing, E., Kim, T., Orso, A., Lee, W., 2018. Enabling refinable cross-host attack investigation with efficient data flow tagging and tracking. In: SEC'18: Proceedings of the 27th USENIX Conference on Security Symposium. USENIX Association, USA, pp. 1705–1722. doi:10.5555/3277203.3277330.

Kaminski, M., Kostylev, E.V., Cuenca Grau, B., 2016. Semantics and expressive power of subqueries and aggregates in SPARQL 1.1. In: Proceedings of the 25th International Conference on World Wide Web, pp. 227–238. https://doi.org/10.1145/2872427.2883022

Kemerlis, V.P., Portokalidis, G., Jee, K., Keromytis, A.D., 2012. libdft: Practical dynamic data flow tracking for commodity systems. SIGPLAN Not 47 (7), 121–132. doi:10.1145/2365864.2151042.

Kenaza, T., Aiash, M., 2016. Toward an efficient ontology-based event correlation in SIEM. Procedia Comput. Sci. 83, 139–146. doi:10.1016/j.procs.2016.04.109.

Kiesling, E., Ekelhart, A., Kurniawan, K., Ekaputra, F., 2019. The SEPSES knowledge graph: an integrated resource for cybersecurity. In: Ghidini, C., Hartig, O., Maleshkova, M., Svátek, V., Cruz, I., Hogan, A., Song, J., Lefrançois, M., Gandon, F. (Eds.), The Semantic Web – ISWC 2019. Springer International Publishing, Cham, pp. 198–214. https://doi.org/10.1007/978-3-030-30796-7_13

King, S.T., Chen, P.M., 2003a. Backtracking intrusions. SIGOPS Oper. Syst. Rev. 37 (5), 223–236. doi:10.1145/1165389.945467.

King, S.T., Chen, P.M., 2003b. Backtracking intrusions. In: SOSP '03: Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles. Association for Computing Machinery, New York, NY, USA, pp. 223–236. doi:10.1145/945445.945467.

Kumar S.. Classification and detection of computer intrusions. 1996. Ph.D. thesis. USA. UMI Order No. GAX96-01522.

Kurniawan, K., Ekelhart, A., Kiesling, E., et al., 2021. An ATT&CK-KG for linking cybersecurity attacks to adversary tactics and techniques. In: The Semantic Web – ISWC 2021, p. 5. http://ceur-ws.org/Vol-2980/paper363.pdf

Kurniawan, K., Ekelhart, A., Kiesling, E., Winkler, D., Quirchmayr, G., Tjoa, A.M., 2022. VloGraph: a virtual knowledge graph framework for distributed security log analysis. Mach. Learn. Knowl. Extr. 4 (2), 371–396. doi:10.3390/make4020016.

Kurniawan, K., Kiesling, E., Ekelhart, A., Ekaputra, F., 2020. Cross-platform file system activity monitoring and forensics a semantic approach. In: Hölbl, M., Rannenberg, K., Welzer, T. (Eds.), ICT Systems Security and Privacy Protection. SEC 2020. IFIP Advances in Information and Communication Technology. Springer, Cham. https://doi.org/10.1007/978-3-030-58201-2_26

Lehmann, J., Sejdiu, G., Bühmann, L., Westphal, P., Stadler, C., Ermilov, I., Bin, S., Chakraborty, N., Saleem, M., Ngomo, A.C.N., et al., 2017. Distributed semantic analytics using the SANSA stack. In: International Semantic Web Conference. Springer, pp. 147–155. https://doi.org/10.1007/978-3-319-68204-4_15

Li, Z., Chen, Q.A., Yang, R., Chen, Y., Ruan, W., 2021. Threat detection and investigation with system-level provenance graphs: a survey. Comput. Secur. 106, 102282. doi:10.1016/j.cose.2021.102282.

Ma, Z., Capretz, M.A.M., Yan, L., 2016. Storing massive resource description framework (RDF) data: a survey. Knowl. Eng. Rev. 31 (4), 391–413. https://doi.org/10.1017/S0269888916000217.

Marz, N., Warren, J., 2015. Big Data: Principles and Best Practices of Scalable Real–Time Data Systems. Manning, Shelter Island, NY. OCLC: ocn909039685

Mcguinness, D.L., Fikes, R., Hendler, J., Stein, L.A., 2002. DAML+OIL: an ontology language for the semantic web. IEEE Intell. Syst. 17 (5), 72–80. doi:10.1109/MIS.2002.1039835.

Milajerdi, S.M., Eshete, B., Gjomemo, R., Venkatakrishnan, V.N., 2019a. POIROT: aligning attack behavior with kernel audit records for cyber threat hunting. In: CCS '19: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. Association for Computing Machinery, New York, NY, USA, pp. 1795–1812. doi:10.1145/3319535.3363217.

Milajerdi, S.M., Gjomemo, R., Eshete, B., Sekar, R., Venkatakrishnan, V.N., et al., 2019b. HOLMES: real-time APT detection through correlation of suspicious information flows. In: 2019 IEEE Symposium on Security and Privacy (SP). IEEE, San Francisco, CA, USA, pp. 1137–1152. doi:10.1109/SP.2019.00026.

Mitre ATT&CK Matrix, 2021Accessed: 30.03.2021, https://attack.mitre.org/matrices/enterprise/.

More, S., Matthews, M., Joshi, A., Finin, T., et al., 2012. A knowledge-based approach to intrusion detection modeling. In: 2012 IEEE Symposium on Security and Privacy Workshops. IEEE, San Francisco, CA, USA, pp. 75–81. doi:10.1109/SPW.2012.26.

Noel, S., Harley, E., Tam, K.H., Limiero, M., Share, M., 2016. Chapter 4 - CyGraph: graph-based analytics and visualization for cybersecurity. In: Gudivada, V.N., Raghavan, V.V., Govindaraju, V., Rao, C.R. (Eds.), Cognitive Computing: Theory and Applications. In: Handbook of Statistics, vol. 35. Elsevier, pp. 117–167. doi:10.1016/bs.host.2016.07.001.

Noy N.F., McGuinness D.L.. Ontology development 101: a guide to creating your first ontology. 2001. http://www.ksl.stanford.edu/KSL_Abstracts/KSL-01-05.html.

Pinkston, J., Undercoffer, J., Joshi, A., Finin, T., 2003. A target-centric ontology for intrusion detection. In: Workshop on Ontologies in Distributed Systems, held at The 18th International Joint Conference on Artificial Intelligence, p. 9. https://www.csee.umbc.edu/~finin/papers/ijcai03OntologiesIDS.pdf

Roth F., Patzke T.. Sigma: generic signature format for SIEM systems, 2021. Accessed: 30.03.2021, https://github.com/SigmaHQ/sigma.

Sarker, I.H., Kayes, A.S.M., Badsha, S., Alqahtani, H., Watters, P., Ng, A., et al., 2020. Cybersecurity data science: an overview from machine learning perspective. J. Big Data 7 (1), 41. doi:10.1186/s40537-020-00318-5.

SPARQL, 20211.1 federated query. Accessed: 30.03.2021, https://www.w3.org/TR/sparql11-federated-query/.

STIX, 2021Structured threat information expression. Accessed: 30.03.2021, https://stixproject.github.io/.

Syed, Z., Padia, A., Finin, T., Mathews, L., Joshi, A., et al., 2016. UCO: a unified cybersecurity ontology. In: AAAI Workshop: Artificial Intelligence for Cyber Security, p. 8. https://www.aaai.org/ocs/index.php/WS/AAAIW16/paper/view/12574/12365

URI Uniform resource identifier, 2021. Accessed: 30.03.2021, https://en.wikipedia.org/wiki/Uniform_Resource_Identifier.

Vargas, H., Buil-Aranda, C., Hogan, A., López, C., 2019. RDF explorer: a visual SPARQL query builder. In: International Semantic Web Conference. Springer, pp. 647–663. https://doi.org/10.1007/978-3-030-30793-6_37

Wylot, M., Hauswirth, M., Cudré-Mauroux, P., Sakr, S., 2018. RDF data storage and query processing schemes: a survey. ACM Comput. Surv. (CSUR) 51 (4), 1–36. https://doi.org/10.1145/3177850.

Xiao, G., Ding, L., Cogrel, B., Calvanese, D., 2019. Virtual knowledge graphs: an overview of systems and use cases. Data Intell. 1 (3), 201–223. https://doi.org/10.1162/dint_a_00011.

Xu, Z., Wu, Z., Li, Z., Jee, K., Rhee, J., Xiao, X., Xu, F., Wang, H., Jiang, G., et al., 2016. High fidelity data reduction for big data security dependency analyses. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. ACM, Vienna Austria, pp. 504–516. doi:10.1145/2976749.2978378.

Zou, Q., Singhal, A., Sun, X., Liu, P., 2020. Automatic recognition of advanced persistent threat tactics for enterprise security. In: IWSPA '20: Proceedings of the Sixth International Workshop on Security and Privacy Analytics. Association for Computing Machinery, New York, NY, USA, pp. 43–52. doi:10.1145/3375708.3380314.

**Kabul Kurniawan** received a BSc and MSc degree in Computer Science at Universitas Gadjah Mada (UGM) Indonesia in 2013 and 2015 respectively. He was a research assistant at the Institute of Software and Information System, Vienna University of Technology in 2018 before joining the Institute of Data, Process, and Knowledge Management as a project assistant at Vienna University of Business and Economics, in 2020. He is currently a Doctoral Candidate at the Computer Science, University of Vienna, Austria. His research including Data Interoperability, Knowledge Management, and Semantic Application in the domain of Cybersecurity in particularly log-stream monitoring and analysis.

**Andreas Ekelhart** is a senior researcher at the Department of Information Systems and Operations at WU and works in the field of IT security with focus on leveraging semantic technologies. He furthermore holds a senior researcher position at the Security and Privacy group at the University of Vienna, and SBA Research, a competence centre for IT Security. He received a master's degree in Business Informatics and a master's degree in Software Engineering & Internet Computing, and holds a Ph.D. in Computer Science from the Institute of Software Technology and Interactive Systems at TU Wien. He is a member of ISC2 and holds various industrial certifications including CISSP, CSSLP, CEH, MCPD, MCSD, and ISTQB CTFL.

**Elmar Kiesling** is a post-doctoral researcher at the Institute for Data, Process and Knowledge Management at WU Wien (Vienna University of Economics and Business), Austria. Furthermore, he coordinates the research area "Data Integration and Analytics for Digital Production" at the Austrian Center for Digital Production (ACDP). Before taking up these positions, he headed the Linked Data Lab at TU Wien (Vienna University of Technology) and was a senior researcher at SBA Research, an industrial research center for IT security, as well as a researcher at the University of Vienna, where he obtained his PhD. His current research focuses on Knowledge Graphs and their industrial applications, most notably in the I4.0 and cybersecurity contexts. Elmar has published numerous publications in these areas and other fields including decision analysis, security management, innovation management, and blended learning.

**Gerald Quirchmayr** holds Doctor's degrees in computer science and law from Johannes Kepler University in Linz (Austria) and currently is Professor in the Multimedia Information Systems Research Group of the Faculty of Computer Science at the University of Vienna. In 2001/2002 he held a Chair in Computer and Information Systems at the University of South Australia. He first joined the University of Vienna in 1993 from the Institute of Computer Science at Johannes Kepler University in Linz (Austria) where he had previously been teaching. His major research focus is on information systems in business and government with a special interest in security, applications, formal representations of decision making and legal issues. His publication record comprises over 200 peer reviewed papers plus several edited books and conference proceedings as well as nationally and internationally published project reports.

**A Min Tjoa** is full professor for Software Technology at the Vienna University of Technology for Software Technology since 1994 and currently also adjunct professor at ITB (Bandung Institute of Technology). He is Vice-President of Infoterm (International Information Center for Terminology) since 2010. Currently, he holds the executive-chairperson position at the Austrian National Competence Center for Excellent Technologies in the field of IT-Security (SBA).He was the Vice-president of the United Nations Commission on Science and Technology for Development. He has been awarded the Honorary Doctorate from the Czech University of Technology in Prague for his research in Data Science.